

Slip 1

Section I: Data Structures

Q 1) C program to create random n elements and sort it in ascending order by using insertion sort.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void insertionSort(int arr[], int n) {
```

```
    int i, key, j;
```

```
    for (i = 1; i < n; i++) {
```

```
        key = arr[i];
```

```
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j + 1] = arr[j];
```

```
            j = j - 1;
```

```
        }
```

```
        arr[j + 1] = key;
```

```
    }
```

```
}
```

```
void printArray(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
  
    int arr[n];  
    srand(time(0));  
    printf("Original array (random elements):\n");  
    for (int i = 0; i < n; i++) {  
        arr[i] = rand() % 100; // Generate random numbers up to 99  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    insertionSort(arr, n);  
  
    printf("Sorted array:\n");  
    printArray(arr, n);  
  
    return 0;  
}
```

OR - Q1) Write a C program to perform the following operations using static implementation of stack: i. Accept n integers from user and push it into the stack. ii. Pop the element from the stack and display it (Use isEmpty()).

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
int isEmpty() {
```

```
    return top == -1;
```

```
}
```

```
void push(int item) {
```

```
    if (top >= MAX - 1) {
```

```
        printf("Stack Overflow!\n");
```

```
        return;
```

```
    }
```

```
    stack[++top] = item;
```

```
}
```

```
int pop() {
```

```
    if (isEmpty()) {
```

```
        printf("Stack Underflow!\n");
```

```
        return -1;
```

```
    }
```

```
    return stack[top--];
```

```
}
```

```
int main() {
```

```
    int n, item;
```

```
    printf("Enter the number of integers to push: ");
```

```

scanf("%d", &n);

printf("Enter %d integers:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &item);
    push(item);
}

printf("Popping elements from the stack:\n");
while (!isEmpty()) {
    printf("%d\n", pop());
}

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the database and insert sufficient records.

-- Database Schema: Room (roomno, room_name, room_type, charges), Guest (Guestcode, Gname, city)

-- Relationship: Room-Guest: one-to-one. room_type can have values as either 'AC' or 'NonAC'.

```

CREATE TABLE Guest (
    Guestcode INT PRIMARY KEY,
    Gname VARCHAR(50),
    city VARCHAR(50)
);

```

```
CREATE TABLE Room (  
    roomno INT PRIMARY KEY,  
    room_name VARCHAR(50),  
    room_type VARCHAR(10) CHECK (room_type IN ('AC', 'NonAC')),  
    charges DECIMAL(10, 2),  
    Guestcode INT UNIQUE,  
    FOREIGN KEY (Guestcode) REFERENCES Guest(Guestcode)  
);
```

-- Insert records

```
INSERT INTO Guest VALUES (1, 'John Doe', 'Pune');
```

```
INSERT INTO Guest VALUES (2, 'Jane Smith', 'Mumbai');
```

```
INSERT INTO Guest VALUES (3, 'Peter Jones', 'Delhi');
```

```
INSERT INTO Guest VALUES (4, 'Susan White', 'Bangalore');
```

```
INSERT INTO Room VALUES (101, 'Suite A', 'AC', 7500.00, 1);
```

```
INSERT INTO Room VALUES (102, 'Deluxe B', 'NonAC', 4500.00, 2);
```

```
INSERT INTO Room VALUES (103, 'Executive C', 'AC', 12000.00, 3);
```

```
INSERT INTO Room VALUES (104, 'Standard D', 'NonAC', 8500.00, 4);
```

```
INSERT INTO Room VALUES (105, 'Standard E', 'AC', 6000.00, NULL);
```

B. Execute the following queries (any 3).

- **i. List the details of the rooms having charges between 5000 and 10000.**

SQL

```
SELECT * FROM Room WHERE charges BETWEEN 5000 AND 10000;
```

- **ii. List the names of the guests in the sorted order by city name.**

SQL

```
SELECT Gname FROM Guest ORDER BY city;
```

- **iii. List the minimum charges of a room.**

SQL

```
SELECT MIN(charges) FROM Room;
```

- **iv. Increase the charges of all AC rooms by 15%.**

SQL

```
UPDATE Room SET charges = charges * 1.15 WHERE room_type = 'AC';
```

- **v. List the names of all the NONAC rooms whose charges are more than 10000.**

SQL

```
SELECT room_name FROM Room WHERE room_type = 'NonAC' AND charges > 10000;
```

C. Write a query to list the name of the guest to whom the room with highest charges is allotted.

SQL

```
SELECT Gname
```

```
FROM Guest
```

```
JOIN Room ON Guest.Guestcode = Room.Guestcode
```

```
WHERE Room.charges = (SELECT MAX(charges) FROM Room);
```

OR C. Create a view to list the names of all the NonAC rooms which have charges greater than at least one of the 'AC' room.

SQL

```
CREATE VIEW NonAC_High_Charges_Rooms AS
```

```
SELECT room_name
```

```
FROM Room
```

```
WHERE room_type = 'NonAC' AND charges > (SELECT MIN(charges) FROM Room  
WHERE room_type = 'AC');
```

Slip 2

Section I: Data Structures

Q.1) Write a C program to implement static Implementation of Queue of integers with following operations: Initialize(), Insert(), Delete(), Isempty(), Display().

C

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX_SIZE 100
```

```
int queue[MAX_SIZE];
```

```
int front = -1, rear = -1;
```

```
void Initialize() {
```

```
    front = -1;
```

```
    rear = -1;
```

```
    printf("Queue initialized.\n");
```

```
}
```

```
bool IsEmpty() {
```

```
    return front == -1;
```

```
}
```

```
void Insert(int value) {
```

```
    if (rear == MAX_SIZE - 1) {
```

```
        printf("Queue is full. Cannot insert.\n");
```

```
    } else {
```

```
        if (front == -1) {
```

```
            front = 0;
```

```
        }
```

```
        rear++;
```

```
        queue[rear] = value;
```

```
    printf("%d inserted into queue.\n", value);
}
}
```

```
void Delete() {
    if (IsEmpty()) {
        printf("Queue is empty. Cannot delete.\n");
    } else {
        int deleted_value = queue[front];
        if (front == rear) {
            front = -1;
            rear = -1;
        } else {
            front++;
        }
        printf("%d deleted from queue.\n", deleted_value);
    }
}
```

```
void Display() {
    if (IsEmpty()) {
        printf("Queue is empty.\n");
    } else {
        printf("Queue elements are: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}
```

```

    }
}

int main() {
    Initialize();
    Insert(10);
    Insert(20);
    Insert(30);
    Display();
    Delete();
    Display();
    return 0;
}

```

OR Q.1) Write a menu driven program for the following operations on singly linked list: i. Create and display a linked list of n nodes. ii. Search a particular element from a linked list.

C

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node* next;
};

```

```

void createList(struct Node** head, int n) {
    struct Node *newNode, *temp;
    int data, i;
    *head = NULL;

```

```

if (n <= 0) {
    printf("List not created.\n");
    return;
}

printf("Enter %d data elements:\n", n);
for (i = 0; i < n; i++) {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        break;
    }
    printf("Enter element %d: ", i + 1);
    scanf("%d", &data);
    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

```

```
}
```

```
void displayList(struct Node* head) {
```

```
    if (head == NULL) {  
        printf("List is empty.\n");  
        return;
```

```
    }
```

```
    printf("Linked list elements: ");
```

```
    struct Node* temp = head;
```

```
    while (temp != NULL) {
```

```
        printf("%d -> ", temp->data);  
        temp = temp->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
void searchElement(struct Node* head, int value) {
```

```
    int position = 1;
```

```
    struct Node* temp = head;
```

```
    while (temp != NULL) {
```

```
        if (temp->data == value) {  
            printf("Element %d found at position %d.\n", value, position);  
            return;
```

```
        }
```

```
        temp = temp->next;
```

```
        position++;
```

```
    }
```

```
    printf("Element %d not found in the list.\n", value);
```

```
}
```

```
int main() {  
    struct Node* head = NULL;  
  
    int choice, n, value;  
  
    do {  
        printf("\n--- Singly Linked List Operations ---\n");  
        printf("1. Create and Display\n");  
        printf("2. Search for an element\n");  
        printf("3. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter the number of nodes: ");  
                scanf("%d", &n);  
                createList(&head, n);  
                displayList(head);  
                break;  
            case 2:  
                if (head == NULL) {  
                    printf("List is empty. Create a list first.\n");  
                    break;  
                }  
                printf("Enter the element to search: ");  
                scanf("%d", &value);
```

```

        searchElement(head, value);
        break;
    case 3:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 3);

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Employee(eno,ename,designation, salary), Department(dno, dname, location)

-- Relationship: Employee-Department: many-to-one.

```

CREATE TABLE Department (
    dno INT PRIMARY KEY,
    dname VARCHAR(50),
    location VARCHAR(50)
);

```

```

CREATE TABLE Employee (

```

```
eno INT PRIMARY KEY,  
ename VARCHAR(50),  
designation VARCHAR(50),  
salary DECIMAL(10, 2),  
dno INT,  
FOREIGN KEY (dno) REFERENCES Department(dno)  
);
```

-- Insert records

```
INSERT INTO Department VALUES (1, 'Sales', 'Pune');
```

```
INSERT INTO Department VALUES (2, 'HR', 'Nashik');
```

```
INSERT INTO Department VALUES (3, 'IT', 'Delhi');
```

```
INSERT INTO Department VALUES (4, 'Marketing', 'Pune');
```

```
INSERT INTO Employee VALUES (101, 'Rahul Sharma', 'Sales Executive', 50000.00, 1);
```

```
INSERT INTO Employee VALUES (102, 'Priya Singh', 'HR Manager', 70000.00, 2);
```

```
INSERT INTO Employee VALUES (103, 'Sanjay Patel', 'Software Engineer', 85000.00, 3);
```

```
INSERT INTO Employee VALUES (104, 'Anjali Roy', 'Marketing Analyst', 45000.00, 4);
```

```
INSERT INTO Employee VALUES (105, 'Amit Gupta', 'HR Executive', 65000.00, 2);
```

```
INSERT INTO Employee VALUES (106, 'Pooja Verma', 'IT Analyst', 60000.00, 3);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Give a 5% raise in salary to all the employees.**

SQL

```
UPDATE Employee SET salary = salary * 1.05;
```

- **ii. Display average salary of an employee.**

SQL

```
SELECT AVG(salary) FROM Employee;
```

- **iii. List the details of all the departments located at 'Nashik' city.**

SQL

```
SELECT * FROM Department WHERE location = 'Nashik';
```

- **iv. Display the details of employees whose names ends with an alphabet 'a'.**

SQL

```
SELECT * FROM Employee WHERE ename LIKE '%a';
```

- **v. Display the location of 'HR' department.**

SQL

```
SELECT location FROM Department WHERE dname = 'HR';
```

C. Write a query to list the details of employees who do not work in any of the departments located at 'Delhi'.

SQL

```
SELECT *
```

```
FROM Employee
```

```
WHERE dno NOT IN (SELECT dno FROM Department WHERE location = 'Delhi');
```

OR C. Create a view to list the names of employees whose salary is greater than all the employees working in 'HR' department.

SQL

```
CREATE VIEW High_Salary_Employees AS
```

```
SELECT ename
```

```
FROM Employee
```

```
WHERE salary > (SELECT MAX(salary) FROM Employee WHERE dno = (SELECT dno  
FROM Department WHERE dname = 'HR'));
```

Slip 3

Section I: Data Structures

Q 1) Accept the array of n integers from user and sort the array in ascending order by using recursive Quick sort algorithm.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void swap(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return (i + 1);
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

```
void printArray(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    srand(time(NULL));
```

```
    printf("Original array (random elements):\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = rand() % 100;
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
    quickSort(arr, 0, n - 1);
```

```
    printf("Sorted array using recursive quick sort:\n");
```

```
    printArray(arr, n);
```

```
    return 0;
```

```
}
```

OR Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int linearSearch(int arr[], int n, int x) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (arr[i] == x) {
```

```
            return i;
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int n, x, result;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    srand(time(NULL));
```

```
    printf("Created random array:\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = rand() % 100;
```

```
        printf("%d ", arr[i]);
```

```

}

printf("\n");

printf("Enter the number to search: ");

scanf("%d", &x);

result = linearSearch(arr, n, x);

if (result != -1) {
    printf("Number %d is present at position %d.\n", x, result + 1);
} else {
    printf("Number %d is not present in the array.\n", x);
}

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Person (pnumber, pname, birthdate, income), Area (aname, area_type, pincode)

-- Relationship: Person-Area: many-to-one. The 'area_type' can have values as either 'urban' or 'rural'.

```

CREATE TABLE Area (
    aname VARCHAR(50) PRIMARY KEY,
    area_type VARCHAR(10) CHECK (area_type IN ('urban', 'rural')),

```

```
    pincode VARCHAR(10)
);
```

```
CREATE TABLE Person (
    pnumber INT PRIMARY KEY,
    pname VARCHAR(50),
    birthdate DATE,
    income DECIMAL(10, 2),
    aname VARCHAR(50),
    FOREIGN KEY (aname) REFERENCES Area(aname)
);
```

-- Insert records

```
INSERT INTO Area VALUES ('Kothrud', 'urban', '411038');
```

```
INSERT INTO Area VALUES ('Camp', 'urban', '411001');
```

```
INSERT INTO Area VALUES ('Shirur', 'rural', '412210');
```

```
INSERT INTO Area VALUES ('Kalyaninagar', 'urban', '411014');
```

```
INSERT INTO Person VALUES (1, 'Ravi Kumar', '1995-07-20', 60000.00, 'Kothrud');
```

```
INSERT INTO Person VALUES (2, 'Priya Singh', '1990-05-15', 75000.00, 'Kothrud');
```

```
INSERT INTO Person VALUES (3, 'Rahul Jain', '1988-07-01', 85000.00, 'Camp');
```

```
INSERT INTO Person VALUES (4, 'Amit Verma', '2001-02-28', 55000.00, 'Shirur');
```

```
INSERT INTO Person VALUES (5, 'Sneha Joshi', '1999-07-10', 90000.00, 'Kalyaninagar');
```

```
INSERT INTO Person VALUES (6, 'Gaurav Kulkarni', '1992-11-05', 120000.00,
'Kalyaninagar');
```

B. Execute the following queries in PostGreSQL (any 3).

- i. List the details of all people whose name starts with the alphabet 'R'.

SQL

SELECT * FROM Person WHERE pname LIKE 'R%';

- **ii. List the names of all people whose birthday falls in the month of 'July'.**

SQL

SELECT pname FROM Person WHERE EXTRACT(MONTH FROM birthdate) = 7;

- **iii. Display the details of people in the sorted order of their income.**

SQL

SELECT * FROM Person ORDER BY income;

- **iv. Display the count of areas of 'urban' type.**

SQL

SELECT COUNT(*) FROM Area WHERE area_type = 'urban';

- **v. Change the pincode of 'kalyaninagar' to 411036.**

SQL

UPDATE Area SET pincode = '411036' WHERE aname = 'Kalyaninagar';

C. Write a query to list the names of people who live in 'Camp' area and have income less than at least one person who lives in 'kalyaninagar' area.

SQL

SELECT pname

FROM Person

WHERE aname = 'Camp'

AND income < (SELECT MAX(income) FROM Person WHERE aname = 'Kalyaninagar');

OR C. Create a view to list the details of the person with second maximum income.

SQL

CREATE VIEW Second_Max_Income_Person AS

SELECT *

FROM Person

ORDER BY income DESC

OFFSET 1

LIMIT 1;

Slip 4

Section I: Data Structures

Q 1) Read the data from file 'student.txt' containing names of student and their class (FY, SY, TY). Accept a student name from user and use linear search algorithm to check whether the name is present in the file and output the Class of student otherwise output "Student not in the list".

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    FILE *file;
```

```
    char name[50], studentName[50], studentClass[10];
```

```
    int found = 0;
```

```
    // Creating a dummy student.txt file for demonstration
```

```
    file = fopen("student.txt", "w");
```

```
    fprintf(file, "John FY\n");
```

```
    fprintf(file, "Jane SY\n");
```

```
    fprintf(file, "Peter TY\n");
```

```
    fclose(file);
```

```
    file = fopen("student.txt", "r");
```

```
    if (file == NULL) {
```

```
        printf("Error opening file.\n");
```

```
        return 1;
```

```
    }
```

```
printf("Enter student name to search: ");
```

```
scanf("%s", name);
```

```
while (fscanf(file, "%s %s", studentName, studentClass) != EOF) {
```

```
    if (strcmp(studentName, name) == 0) {
```

```
        printf("Student found. Class: %s\n", studentClass);
```

```
        found = 1;
```

```
        break;
```

```
    }
```

```
}
```

```
if (!found) {
```

```
    printf("Student not in the list.\n");
```

```
}
```

```
fclose(file);
```

```
return 0;
```

```
}
```

OR Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 100
```

```
char stack[MAX_SIZE];
```

```
int top = -1;
```

```
void push(char c) {  
    if (top < MAX_SIZE - 1) {  
        stack[++top] = c;  
    } else {  
        printf("Stack overflow!\n");  
    }  
}
```

```
char pop() {  
    if (top != -1) {  
        return stack[top--];  
    } else {  
        return '\0';  
    }  
}
```

```
int main() {  
    char str[MAX_SIZE];  
    printf("Enter a string: ");  
    scanf("%s", str);  
  
    for (int i = 0; i < strlen(str); i++) {  
        push(str[i]);  
    }  
  
    printf("Reversed string: ");  
    while (top != -1) {
```

```
    printf("%c", pop());  
}  
printf("\n");  
  
return 0;  
}
```

Section II: Database Management Systems

Q.2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Policy (pno, pname, premium_amt, policy_type), Customer (cno, cname, city, agent_name)

-- Relationship: Policy-Customer: many-to-one. The 'policy_type' can have values as 'Yearly', 'Half-yearly' or 'Monthly'.

```
CREATE TABLE Customer (  
    cno INT PRIMARY KEY,  
    cname VARCHAR(50),  
    city VARCHAR(50),  
    agent_name VARCHAR(50)  
);
```

```
CREATE TABLE Policy (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(50),  
    premium_amt DECIMAL(10, 2),  
    policy_type VARCHAR(20) CHECK (policy_type IN ('Yearly', 'Half-yearly', 'Monthly')),  
    cno INT,
```

```
FOREIGN KEY (cno) REFERENCES Customer(cno)
);
```

-- Insert records

```
INSERT INTO Customer VALUES (1, 'Rajesh Kumar', 'Pune', 'Agent A');
INSERT INTO Customer VALUES (2, 'Shweta Patil', 'Mumbai', 'Agent B');
INSERT INTO Customer VALUES (3, 'Amit Sharma', 'Delhi', 'Agent A');
INSERT INTO Customer VALUES (4, 'Pooja Gupta', 'Pune', 'Agent C');
```

```
INSERT INTO Policy VALUES (101, 'Jeevan Anand', 25000.00, 'Yearly', 1);
INSERT INTO Policy VALUES (102, 'Money Back', 1200.00, 'Monthly', 2);
INSERT INTO Policy VALUES (103, 'Term Plan', 50000.00, 'Half-yearly', 1);
INSERT INTO Policy VALUES (104, 'Health Plus', 800.00, 'Monthly', 3);
INSERT INTO Policy VALUES (105, 'Jeevan Anand', 30000.00, 'Yearly', 4);
```

B. Execute the following queries in PostgreSQL (any 3).

- **i. List the details of all customers who live in city.** (Assuming it means list all customers)

SQL

```
SELECT * FROM Customer;
```

- **ii. Display the average premium amount.**

SQL

```
SELECT AVG(premium_amt) FROM Policy;
```

- **iii. Count the number of customers who have taken 'Jeevan Anand' policy.**

SQL

```
SELECT COUNT(DISTINCT cno) FROM Policy WHERE pname = 'Jeevan Anand';
```

- **iv. Increase the premium amount for 'Monthly' policies by 10%.**

SQL

```
UPDATE Policy SET premium_amt = premium_amt * 1.10 WHERE policy_type = 'Monthly';
```

- **v. Display the policy_type wise count of policies.**

SQL

```
SELECT policy_type, COUNT(*) FROM Policy GROUP BY policy_type;
```

C. Write a query to list the policy_type with highest average premium amount.

SQL

```
SELECT policy_type  
FROM Policy  
GROUP BY policy_type  
ORDER BY AVG(premium_amt) DESC  
LIMIT 1;
```

OR C. Create a view to list the name of the customer who has taken highest number of policies.

SQL

```
CREATE VIEW Top_Customer AS  
SELECT C.cname  
FROM Customer C  
JOIN Policy P ON C.cno = P.cno  
GROUP BY C.cno, C.cname  
ORDER BY COUNT(P.pno) DESC  
LIMIT 1;
```

Slip 5

Section I: Data Structures

Q 1) Write a C program to implement singly linked list with Create and Display operation.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node* create(int n) {  
    struct Node* head = NULL;  
    struct Node* temp = NULL;  
    struct Node* p = NULL;  
    int data;
```

```
    for (int i = 0; i < n; i++) {  
        printf("Enter element %d: ", i + 1);  
        scanf("%d", &data);
```

```
        temp = (struct Node*)malloc(sizeof(struct Node));  
        temp->data = data;  
        temp->next = NULL;
```

```
        if (head == NULL) {  
            head = temp;  
        } else {  
            p = head;  
            while (p->next != NULL) {  
                p = p->next;  
            }  
            p->next = temp;  
        }  
    }
```

```

    }

    return head;
}

void display(struct Node* head) {
    if (head == NULL) {
        printf("Linked list is empty.\n");
        return;
    }
    printf("Linked list elements: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n;

    printf("Enter the number of nodes to create: ");
    scanf("%d", &n);

    struct Node* list = create(n);

    display(list);

    return 0;
}

```

OR Q 1) Read the data from file 'student.txt' containing names of student and their class (FY,SY,TY). Accept a student name from user and use binary search algorithm to check whether the name is present in the file and output the class of student otherwise output "Student not in the list".

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    char name[50];
```

```
    char class[10];
```

```
} Student;
```

```
int compareStudents(const void* a, const void* b) {
```

```
    return strcmp(((Student*)a)->name, ((Student*)b)->name);
```

```
}
```

```
int binarySearch(Student students[], int n, char* name) {
```

```
    int low = 0, high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
```

```
        int cmp = strcmp(students[mid].name, name);
```

```
        if (cmp == 0) {
```

```
            return mid;
```

```
        }
```

```
        if (cmp < 0) {
```

```
            low = mid + 1;
```

```
        } else {
```

```
        high = mid - 1;
    }
}
return -1;
}
```

```
int main() {
    FILE *file;

    Student students[100];

    int count = 0, index;

    char searchName[50];

    // Create a dummy file with sorted data for binary search

    file = fopen("student.txt", "w");
    fprintf(file, "Alex FY\n");
    fprintf(file, "Charlie SY\n");
    fprintf(file, "David TY\n");
    fprintf(file, "Grace FY\n");
    fclose(file);

    file = fopen("student.txt", "r");
    if (file == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    while (fscanf(file, "%s %s", students[count].name, students[count].class) == 2) {
        count++;
    }
}
```

```

}

fclose(file);

// Sort the array in memory
qsort(students, count, sizeof(Student), compareStudents);

printf("Enter student name to search: ");
scanf("%s", searchName);

index = binarySearch(students, count, searchName);

if (index != -1) {
    printf("Student found. Class: %s\n", students[index].class);
} else {
    printf("Student not in the list.\n");
}

return 0;
}

```

Section II: Database Management Systems

Q.2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Doctor (dno, dname, city), Patient (pno, pat_name, city, disease)

-- Relationship: Doctor-Patient: many-to-many with Fees as a descriptive attribute.

CREATE TABLE Doctor (

dno INT PRIMARY KEY,

```
    dname VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Patient (  
    pno INT PRIMARY KEY,  
    pat_name VARCHAR(50),  
    city VARCHAR(50),  
    disease VARCHAR(50)  
);
```

```
CREATE TABLE Doctor_Patient (  
    dno INT,  
    pno INT,  
    fees DECIMAL(10, 2),  
    PRIMARY KEY (dno, pno),  
    FOREIGN KEY (dno) REFERENCES Doctor(dno),  
    FOREIGN KEY (pno) REFERENCES Patient(pno)  
);
```

-- Insert records

```
INSERT INTO Doctor VALUES (1, 'Dr. Patil', 'Pune');
```

```
INSERT INTO Doctor VALUES (2, 'Dr. Kumar', 'Nagpur');
```

```
INSERT INTO Doctor VALUES (3, 'Dr. Sharma', 'Pune');
```

```
INSERT INTO Patient VALUES (101, 'Amit Singh', 'Pune', 'Fever');
```

```
INSERT INTO Patient VALUES (102, 'Priya Mehta', 'Nagpur', 'Asthama');
```

```
INSERT INTO Patient VALUES (103, 'Rajesh Kumar', 'Mumbai', 'Cancer');
```

```
INSERT INTO Patient VALUES (104, 'Aniket Jadhav', 'Pune', 'Asthama');
```

```
INSERT INTO Patient VALUES (105, 'Dr. Patil', 'Pune', 'Fever');
```

```
INSERT INTO Doctor_Patient VALUES (1, 101, 300.00);
```

```
INSERT INTO Doctor_Patient VALUES (2, 102, 200.00);
```

```
INSERT INTO Doctor_Patient VALUES (1, 104, 250.00);
```

```
INSERT INTO Doctor_Patient VALUES (2, 104, 200.00);
```

```
INSERT INTO Doctor_Patient VALUES (3, 103, 500.00);
```

B. Execute the following queries in PostGreSQL(any 3).

- **i. List the details of all the doctors from 'Nagpur'city.**

SQL

```
SELECT * FROM Doctor WHERE city = 'Nagpur';
```

- **ii. Display count of patients.**

SQL

```
SELECT COUNT(*) FROM Patient;
```

- **iii. List the names of the patients suffering from 'cancer' disease.**

SQL

```
SELECT pat_name FROM Patient WHERE disease = 'Cancer';
```

- **iv. Change the city of 'Dr. Patil' to Pune.** (The sample data already has Dr. Patil in Pune, so this query won't change anything, but the syntax is correct).

SQL

```
UPDATE Doctor SET city = 'Pune' WHERE dname = 'Dr. Patil';
```

- **v. List the names of patients that starts with alphabet 'A'.**

SQL

```
SELECT pat_name FROM Patient WHERE pat_name LIKE 'A%';
```

C. Write a query to find the number of patients suffering from "Asthama" and have been charged a fee of Rs. 200 by "Dr. Kumar".

SQL

```
SELECT COUNT(*)
```

```
FROM Patient p
JOIN Doctor_Patient dp ON p.pno = dp.pno
JOIN Doctor d ON dp.dno = d.dno
WHERE p.disease = 'Asthama' AND dp.fees = 200 AND d.dname = 'Dr. Kumar';
```

OR C. Create a view to find the name of a doctor who treats maximum number of patients.

SQL

```
CREATE VIEW Top_Doctor AS
SELECT d.dname
FROM Doctor d
JOIN Doctor_Patient dp ON d.dno = dp.dno
GROUP BY d.dno, d.dname
ORDER BY COUNT(dp.pno) DESC
LIMIT 1;
```

Slip 6

Section I: Data Structures

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
void swap(int *xp, int *yp) {
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}
```

```
void selectionSort(int arr[], int n) {  
    int i, j, min_idx;  
    for (i = 0; i < n - 1; i++) {  
        min_idx = i;  
        for (j = i + 1; j < n; j++) {  
            if (arr[j] < arr[min_idx]) {  
                min_idx = j;  
            }  
        }  
        if (min_idx != i) {  
            swap(&arr[min_idx], &arr[i]);  
        }  
    }  
}
```

```
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
    int arr[n];
```

```

srand(time(NULL));

printf("Original random array:\n");
for (int i = 0; i < n; i++) {
    arr[i] = rand() % 100;
    printf("%d ", arr[i]);
}
printf("\n");

selectionSort(arr, n);
printf("Sorted array:\n");
printArray(arr, n);

return 0;
}

```

OR Q 1) Write a menu driven program for the following operations on singly link list:
i. Create and display a linked list of n nodes. ii. Search an element in a linked list.

C

```

#include <stdio.h>

#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

void createList(struct Node** head, int n) {
    struct Node *newNode, *temp;

```

```
int data, i;

*head = NULL;

if (n <= 0) {

    printf("List not created.\n");

    return;

}

printf("Enter %d data elements:\n", n);

for (i = 0; i < n; i++) {

    newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {

        printf("Memory allocation failed.\n");

        break;

    }

    printf("Enter element %d: ", i + 1);

    scanf("%d", &data);

    newNode->data = data;

    newNode->next = NULL;

    if (*head == NULL) {

        *head = newNode;

    } else {

        temp = *head;

        while (temp->next != NULL) {

            temp = temp->next;

        }

        temp->next = newNode;

    }

}
```

```
    }  
}  
}
```

```
void displayList(struct Node* head) {  
    if (head == NULL) {  
        printf("List is empty.\n");  
        return;  
    }  
    printf("Linked list elements: ");  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
void searchElement(struct Node* head, int value) {  
    int position = 1;  
    struct Node* temp = head;  
    while (temp != NULL) {  
        if (temp->data == value) {  
            printf("Element %d found at position %d.\n", value, position);  
            return;  
        }  
        temp = temp->next;  
        position++;  
    }  
}
```

```
}  
  
printf("Element %d not found in the list.\n", value);  
}
```

```
int main() {  
  
    struct Node* head = NULL;  
  
    int choice, n, value;  
  
    do {  
  
        printf("\n--- Singly Linked List Operations ---\n");  
  
        printf("1. Create and Display\n");  
  
        printf("2. Search for an element\n");  
  
        printf("3. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
  
        switch (choice) {  
  
            case 1:  
  
                printf("Enter the number of nodes: ");  
  
                scanf("%d", &n);  
  
                createList(&head, n);  
  
                displayList(head);  
  
                break;  
  
            case 2:  
  
                if (head == NULL) {  
  
                    printf("List is empty. Create a list first.\n");  
  
                    break;  
  
                }  
  
            }
```

```

        printf("Enter the element to search: ");
        scanf("%d", &value);
        searchElement(head, value);
        break;
    case 3:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 3);

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Student (rno,name,city), Teacher(tno,tname,phone_no,salary)

-- Relationship: Student-Teacher: many-to-many with subject as a descriptive attribute.

```

CREATE TABLE Student (
    rno INT PRIMARY KEY,
    name VARCHAR(50),
    city VARCHAR(50)
);

```

```
CREATE TABLE Teacher (  
    tno INT PRIMARY KEY,  
    tname VARCHAR(50),  
    phone_no VARCHAR(15),  
    salary DECIMAL(10, 2)  
);
```

```
CREATE TABLE Student_Teacher (  
    rno INT,  
    tno INT,  
    subject VARCHAR(50),  
    PRIMARY KEY (rno, tno, subject),  
    FOREIGN KEY (rno) REFERENCES Student(rno),  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

-- Insert records

```
INSERT INTO Student VALUES (1, 'Rahul Sharma', 'Ahilyanagar');
```

```
INSERT INTO Student VALUES (2, 'Priya Singh', 'Pune');
```

```
INSERT INTO Student VALUES (3, 'Rajesh Patil', 'Ahilyanagar');
```

```
INSERT INTO Student VALUES (4, 'Amit Verma', 'Mumbai');
```

```
INSERT INTO Teacher VALUES (101, 'Mr. Patil', '9822131225', 45000.00);
```

```
INSERT INTO Teacher VALUES (102, 'Mrs. Joshi', '9988776655', 55000.00);
```

```
INSERT INTO Teacher VALUES (103, 'Mr. Rao', '9000011111', 65000.00);
```

```
INSERT INTO Teacher VALUES (104, 'Mr. Reddy', '9876543210', 48000.00);
```

```
INSERT INTO Student_Teacher VALUES (1, 101, 'DBMS');
```

```
INSERT INTO Student_Teacher VALUES (2, 101, 'Data Structures');
```

```
INSERT INTO Student_Teacher VALUES (3, 102, 'DBMS');
```

```
INSERT INTO Student_Teacher VALUES (4, 103, 'Operating Systems');
```

```
INSERT INTO Student_Teacher VALUES (1, 104, 'DBMS');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. List the names of students from 'Ahilyanagar' city.**

SQL

```
SELECT name FROM Student WHERE city = 'Ahilyanagar';
```

- **ii. Display the count of teachers whose salary is between 40000 to 60000.**

SQL

```
SELECT COUNT(*) FROM Teacher WHERE salary BETWEEN 40000 AND 60000;
```

- **iii. Change the phone number of 'Prof. Patil' to '9822131226'.**

SQL

```
UPDATE Teacher SET phone_no = '9822131226' WHERE tname = 'Mr. Patil';
```

- **iv. List the details of the teachers in the sorted order of their name.**

SQL

```
SELECT * FROM Teacher ORDER BY tname;
```

- **v. List the names of the teachers who have salary less than 50000.**

SQL

```
SELECT tname FROM Teacher WHERE salary < 50000;
```

C. Write a query to list the names of all teachers who do not teach any subject that is taught by 'Mr. Patil'.

SQL

```
SELECT tname
```

```
FROM Teacher
```

```
WHERE tno NOT IN (
```

```
    SELECT tno
```

```
    FROM Student_Teacher
```

```
WHERE subject IN (  
    SELECT subject  
    FROM Student_Teacher st  
    JOIN Teacher t ON st.tno = t.tno  
    WHERE t.tname = 'Mr. Patil'  
)  
);
```

OR C. Create a view to list the details of students who have taken 'DBMS' subject.

SQL

```
CREATE VIEW DBMS_Students AS  
SELECT s.rno, s.name, s.city  
FROM Student s  
JOIN Student_Teacher st ON s.rno = st.rno  
WHERE st.subject = 'DBMS';
```

Slip 7

Section I: Data Structures

Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

C

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int x) {  
    int low = 0;  
    int high = n - 1;  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] == x) {
```

```
        return mid;
    }
    if (arr[mid] < x) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
return -1;
}
```

```
int main() {
    int n, x, result;
    printf("Enter the number of sorted elements: ");
    scanf("%d", &n);
    int arr[n];

    printf("Enter %d sorted integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the value to search: ");
    scanf("%d", &x);

    result = binarySearch(arr, n, x);

    if (result != -1) {
```

```

        printf("Number %d is present at position %d.\n", x, result + 1);
    } else {
        printf("Number %d is not present in the array.\n", x);
    }

    return 0;
}

```

OR Q 1) Read the data from "Student.txt (Stud_name, Age, Percentage)" file and sort on Age using insertion sort. (Create a separate sorted file).

C

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

```

```

typedef struct {
    char name[50];
    int age;
    float percentage;
} Student;

```

```

void insertionSort(Student arr[], int n) {
    int i, j;
    Student key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j].age > key.age) {
            arr[j + 1] = arr[j];

```

```

        j = j - 1;
    }
    arr[j + 1] = key;
}
}

```

```

int main() {
    FILE *in_file, *out_file;

    Student students[100];

    int count = 0;

    // Create a dummy input file
    in_file = fopen("Student.txt", "w");
    fprintf(in_file, "John 22 85.5\n");
    fprintf(in_file, "Jane 20 90.0\n");
    fprintf(in_file, "Peter 25 78.9\n");
    fprintf(in_file, "Alex 21 92.1\n");
    fclose(in_file);

    in_file = fopen("Student.txt", "r");
    if (in_file == NULL) {
        printf("Error opening input file.\n");
        return 1;
    }

    while (fscanf(in_file, "%s %d %f", students[count].name, &students[count].age,
&students[count].percentage) == 3) {
        count++;
    }
}

```

```

    }

    fclose(in_file);

    insertionSort(students, count);

    out_file = fopen("sortedemponage.txt", "w");
    if (out_file == NULL) {
        printf("Error opening output file.\n");
        return 1;
    }

    printf("Sorted data by age:\n");
    for (int i = 0; i < count; i++) {
        printf("%s %d %.2f\n", students[i].name, students[i].age, students[i].percentage);
        fprintf(out_file, "%s %d %.2f\n", students[i].name, students[i].age,
students[i].percentage);
    }
    fclose(out_file);

    return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Item (itemno, name, quantity), Supplier (sno, name, city)

-- Relationship: Item-Supplier: many-to-many with rate as a descriptive attribute.

```
CREATE TABLE Item (  
    itemno INT PRIMARY KEY,  
    name VARCHAR(50),  
    quantity INT  
);
```

```
CREATE TABLE Supplier (  
    sno INT PRIMARY KEY,  
    name VARCHAR(50),  
    city VARCHAR(50)  
);
```

```
CREATE TABLE Item_Supplier (  
    itemno INT,  
    sno INT,  
    rate DECIMAL(10, 2),  
    PRIMARY KEY (itemno, sno),  
    FOREIGN KEY (itemno) REFERENCES Item(itemno),  
    FOREIGN KEY (sno) REFERENCES Supplier(sno)  
);
```

-- Insert records

```
INSERT INTO Item VALUES (1, 'Mouse', 50);  
INSERT INTO Item VALUES (2, 'Keyboard', 30);  
INSERT INTO Item VALUES (3, 'Monitor', 15);  
INSERT INTO Item VALUES (4, 'Pendrive', 5);
```

```
INSERT INTO Supplier VALUES (101, 'Mahesh Traders', 'Pune');  
INSERT INTO Supplier VALUES (102, 'Manoj Suppliers', 'Nagpur');  
INSERT INTO Supplier VALUES (103, 'Patel Electronics', 'Mumbai');
```

```
INSERT INTO Item_Supplier VALUES (1, 101, 500.00);  
INSERT INTO Item_Supplier VALUES (1, 102, 550.00);  
INSERT INTO Item_Supplier VALUES (2, 101, 1200.00);  
INSERT INTO Item_Supplier VALUES (3, 103, 8000.00);  
INSERT INTO Item_Supplier VALUES (4, 103, 450.00);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Change the quantity of item 'Mouse' to 80.**

SQL

```
UPDATE Item SET quantity = 80 WHERE name = 'Mouse';
```

- **ii. List the details of the suppliers whose name begins with the alphabet 'M'.**

SQL

```
SELECT * FROM Supplier WHERE name LIKE 'M%';
```

- **iii. Display the total count of items.**

SQL

```
SELECT SUM(quantity) FROM Item;
```

- **iv. List the names of suppliers who do not live in Pune city.**

SQL

```
SELECT name FROM Supplier WHERE city != 'Pune';
```

- **v. List the names of items with quantity less than 10.**

SQL

```
SELECT name FROM Item WHERE quantity < 10;
```

C. Write a query to display the name of supplier who supplies Item with minimum rate.

SQL

```
SELECT s.name
FROM Supplier s
JOIN Item_Supplier is_link ON s.sno = is_link.sno
WHERE is_link.rate = (SELECT MIN(rate) FROM Item_Supplier);
```

OR C. Create a view to display supplier-wise list of items.

SQL

```
CREATE VIEW Supplier_Items AS
SELECT s.name AS supplier_name, i.name AS item_name
FROM Supplier s
JOIN Item_Supplier is_link ON s.sno = is_link.sno
JOIN Item i ON is_link.itemno = i.itemno
ORDER BY s.name;
```

Slip 8

Section I: Data Structures

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void swap(int *xp, int *yp) {
```

```
    int temp = *xp;
```

```
    *xp = *yp;
```

```
    *yp = temp;
```

```
}
```

```

void bubbleSort(int arr[], int n) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

```

```

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(NULL));

    printf("Original random array:\n");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
    }
}

```

```

        printf("%d ", arr[i]);
    }
    printf("\n");

    bubbleSort(arr, n);
    printf("Sorted array:\n");
    printArray(arr, n);

    return 0;
}

```

OR Q 1) Write a menu driven program for the following operations on doubly link list: i. Create and Display a link list of n nodes. ii. Insert an element in a linked list in a particular position.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

```

```

struct Node* createList(int n) {
    struct Node* head = NULL;
    struct Node* newNode, *temp;
    int data, i;

    if (n <= 0) {

```

```

printf("List not created.\n");

return NULL;

}

printf("Enter %d data elements:\n", n);
for (i = 0; i < n; i++) {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        break;
    }
    printf("Enter element %d: ", i + 1);
    scanf("%d", &data);
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}
}

```

```
    return head;
}
```

```
void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Doubly linked list elements (forward): ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```
void insertAtPosition(struct Node** head, int data, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (position == 1) {
```

```

newNode->next = *head;
if (*head != NULL) {
    (*head)->prev = newNode;
}
*head = newNode;
return;
}

```

```

struct Node* temp = *head;
int i = 1;
while (i < position - 1 && temp != NULL) {
    temp = temp->next;
    i++;
}

```

```

if (temp == NULL) {
    printf("Invalid position.\n");
    free(newNode);
    return;
}

```

```

newNode->next = temp->next;
newNode->prev = temp;
if (temp->next != NULL) {
    temp->next->prev = newNode;
}
temp->next = newNode;
}

```

```
int main() {  
    struct Node* head = NULL;  
    int choice, n, data, position;  
  
    do {  
        printf("\n--- Doubly Linked List Operations ---\n");  
        printf("1. Create and Display\n");  
        printf("2. Insert at a particular position\n");  
        printf("3. Exit\n");  
        printf("Enter your choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter the number of nodes: ");  
                scanf("%d", &n);  
                head = createList(n);  
                displayList(head);  
                break;  
            case 2:  
                printf("Enter the element to insert: ");  
                scanf("%d", &data);  
                printf("Enter the position to insert: ");  
                scanf("%d", &position);  
                insertAtPosition(&head, data, position);  
                displayList(head);  
                break;
```

```

        case 3:
            printf("Exiting program.\n");
            break;
        default:
            printf("Invalid choice. Please try again.\n");
    }
} while (choice != 3);

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

```

-- Database Schema: Student (sno, s_name, s_class), Teacher (tno, t_name,
yrs_experience)

-- Relationship: Student-Teacher: Many to Many with Subject as a descriptive attribute.

-- Constraint: s_class should be either "FY", "SY" or "TY"

```

```

CREATE TABLE Student (
    sno INT PRIMARY KEY,
    s_name VARCHAR(50),
    s_class VARCHAR(10) CHECK (s_class IN ('FY', 'SY', 'TY'))
);

```

```

CREATE TABLE Teacher (
    tno INT PRIMARY KEY,

```

```
t_name VARCHAR(50),  
yrs_experience INT  
);
```

```
CREATE TABLE Student_Teacher (  
    sno INT,  
    tno INT,  
    subject VARCHAR(50),  
    PRIMARY KEY (sno, tno, subject),  
    FOREIGN KEY (sno) REFERENCES Student(sno),  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

-- Insert records

```
INSERT INTO Student VALUES (101, 'Rahul', 'FY');  
INSERT INTO Student VALUES (102, 'Priya', 'SY');  
INSERT INTO Student VALUES (103, 'Sanjay', 'TY');  
INSERT INTO Student VALUES (104, 'Nikhil', 'SY');  
INSERT INTO Student VALUES (105, 'Sagar', 'TY');
```

```
INSERT INTO Teacher VALUES (1, 'Dr. Wani', 10);  
INSERT INTO Teacher VALUES (2, 'Mrs. Deshmukh', 5);  
INSERT INTO Teacher VALUES (3, 'Mr. Jadhav', 15);  
INSERT INTO Teacher VALUES (4, 'Mr. Patil', 8);
```

```
INSERT INTO Student_Teacher VALUES (101, 1, 'Data Structures');  
INSERT INTO Student_Teacher VALUES (102, 2, 'Maths');  
INSERT INTO Student_Teacher VALUES (103, 3, 'Maths');
```

```
INSERT INTO Student_Teacher VALUES (104, 1, 'Maths');
```

```
INSERT INTO Student_Teacher VALUES (105, 4, 'DBMS');
```

```
INSERT INTO Student_Teacher VALUES (101, 2, 'DBMS');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Give class-wise number of students.**

SQL

```
SELECT s_class, COUNT(*) FROM Student GROUP BY s_class;
```

- **ii. List all students studying in class "TY".**

SQL

```
SELECT * FROM Student WHERE s_class = 'TY';
```

- **iii. Count the number of students who have taken subject "Maths".**

SQL

```
SELECT COUNT(DISTINCT sno) FROM Student_Teacher WHERE subject = 'Maths';
```

- **iv. Delete record of student whose sno "105".**

SQL

```
DELETE FROM Student_Teacher WHERE sno = 105;
```

```
DELETE FROM Student WHERE sno = 105;
```

- **v. Add designation column in teacher table.**

SQL

```
ALTER TABLE Teacher ADD COLUMN designation VARCHAR(50);
```

C. Write a query to list the names of all teachers with their subjects along with the total number of students they are teaching.

SQL

```
SELECT T.t_name, ST.subject, COUNT(ST.sno) AS total_students
```

```
FROM Teacher T
```

```
JOIN Student_Teacher ST ON T.tno = ST.tno
```

```
GROUP BY T.t_name, ST.subject
```

```
ORDER BY T.t_name;
```

OR C. Create a view to find the teacher who teaches maximum number of subjects.

SQL

```
CREATE VIEW Teacher_Max_Subjects AS
```

```
SELECT t.t_name
```

```
FROM Teacher t
```

```
JOIN Student_Teacher st ON t.tno = st.tno
```

```
GROUP BY t.t_name
```

```
ORDER BY COUNT(st.subject) DESC
```

```
LIMIT 1;
```

Slip 9

Section I: Data Structures

Q 1) Write a menu driven program for the following operations on singly linked list:

iii. Create and display a linked list of n nodes. iv. Delete a particular element from a linked list.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* createList(int n) {
```

```
    struct Node* head = NULL;
```

```
    struct Node* newNode, *temp;
```

```
    int data;
```

```

if (n <= 0) return NULL;

printf("Enter %d data elements:\n", n);
for (int i = 0; i < n; i++) {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) break;
    printf("Enter element %d: ", i + 1);
    scanf("%d", &data);
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
return head;
}

```

```

void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
}

```

```

}

printf("Linked list elements: ");

struct Node* temp = head;

while (temp != NULL) {

    printf("%d -> ", temp->data);

    temp = temp->next;

}

printf("NULL\n");

}

void deleteElement(struct Node** head, int key) {

    struct Node *temp = *head, *prev = NULL;

    if (temp != NULL && temp->data == key) {

        *head = temp->next;

        free(temp);

        printf("Element %d deleted.\n", key);

        return;

    }

    while (temp != NULL && temp->data != key) {

        prev = temp;

        temp = temp->next;

    }

    if (temp == NULL) {

        printf("Element %d not found.\n", key);

        return;

```

```
}
```

```
prev->next = temp->next;
```

```
free(temp);
```

```
printf("Element %d deleted.\n", key);
```

```
}
```

```
int main() {
```

```
    struct Node* head = NULL;
```

```
    int choice, n, element;
```

```
    do {
```

```
        printf("\n--- Singly Linked List Operations ---\n");
```

```
        printf("1. Create and Display\n");
```

```
        printf("2. Delete a particular element\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1:
```

```
                printf("Enter the number of nodes: ");
```

```
                scanf("%d", &n);
```

```
                head = createList(n);
```

```
                displayList(head);
```

```
                break;
```

```
            case 2:
```

```
                if (head == NULL) {
```

```

        printf("List is empty. Create a list first.\n");
        break;
    }
    printf("Enter the element to delete: ");
    scanf("%d", &element);
    deleteElement(&head, element);
    displayList(head);
    break;
case 3:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice. Please try again.\n");
}
} while (choice != 3);

return 0;
}

```

OR Q 1) Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

C

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

```

int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {

```

```
        return i;
    }
}
return -1;
}
```

```
int main() {
    int n, x, result;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(NULL));

    printf("Created random array:\n");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    printf("Enter the number to search: ");
    scanf("%d", &x);

    result = linearSearch(arr, n, x);

    if (result != -1) {
        printf("Number %d is present at position %d.\n", x, result + 1);
    } else {
```

```
        printf("Number %d is not present in the array.\n", x);
    }

    return 0;
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Person (pnumber, pname, birthdate, income),
Area(aname,area_type).

-- Relationship: Area and Person are in a one-to-many relationship.

-- Constraint: 'area_type' should have value either urban or rural.

```
CREATE TABLE Area (  
    aname VARCHAR(50) PRIMARY KEY,  
    area_type VARCHAR(10) CHECK (area_type IN ('urban', 'rural'))  
);
```

```
CREATE TABLE Person (  
    pnumber INT PRIMARY KEY,  
    pname VARCHAR(50),  
    birthdate DATE,  
    income DECIMAL(10, 2),  
    aname VARCHAR(50),  
    FOREIGN KEY (aname) REFERENCES Area(aname)  
);
```

-- Insert records

```
INSERT INTO Area VALUES ('Pune', 'urban');
```

```
INSERT INTO Area VALUES ('Delhi', 'urban');
```

```
INSERT INTO Area VALUES ('Shirur', 'rural');
```

```
INSERT INTO Person VALUES (101, 'Amit Shah', '1990-02-15', 50000.00, 'Pune');
```

```
INSERT INTO Person VALUES (102, 'Bhavana Roy', '1985-06-20', 120000.00, 'Pune');
```

```
INSERT INTO Person VALUES (103, 'Chetan B', '1992-02-28', 80000.00, 'Delhi');
```

```
INSERT INTO Person VALUES (104, 'Dinesh K', '1998-10-10', 45000.00, 'Shirur');
```

```
INSERT INTO Person VALUES (105, 'Anil S', '1991-03-05', 15000.00, 'Pune');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. List the details of persons with income greater than 10000.**

SQL

```
SELECT * FROM Person WHERE income > 10000;
```

- **ii. Transfer all people living in 'Pune' to 'Delhi'.** (This requires a change in the foreign key, so first ensure 'Delhi' exists).

SQL

```
UPDATE Person SET aname = 'Delhi' WHERE aname = 'Pune';
```

- **iii. Count number of area's which has type 'urban'.**

SQL

```
SELECT COUNT(*) FROM Area WHERE area_type = 'urban';
```

- **iv. Give the count of people who are born in the month of February.**

SQL

```
SELECT COUNT(*) FROM Person WHERE EXTRACT(MONTH FROM birthdate) = 2;
```

- **v. List the names of all people living in 'rural' area.**

SQL

```
SELECT P.pname
```

```
FROM Person P
JOIN Area A ON P.aname = A.aname
WHERE A.area_type = 'rural';
```

C. Write a query that count the total number of people staying in "urban" area with income equal to average income.

SQL

```
SELECT COUNT(*)
FROM Person P
JOIN Area A ON P.aname = A.aname
WHERE A.area_type = 'urban' AND P.income = (SELECT AVG(income) FROM Person);
```

OR C. Create a view to print the person's details along with their area name.

SQL

```
CREATE VIEW Person_Area_Details AS
SELECT p.pnumber, p.pname, p.birthdate, p.income, a.aname
FROM Person p
JOIN Area a ON p.aname = a.aname;
```

Slip 10

Section I: Data Structures

Q1) Create random array of n integers and sort the array in ascending order by using recursive Merge sort algorithm.

C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
```

```
int n2 = r - m;
```

```
int L[n1], R[n2];
```

```
for (i = 0; i < n1; i++)
```

```
    L[i] = arr[l + i];
```

```
for (j = 0; j < n2; j++)
```

```
    R[j] = arr[m + 1 + j];
```

```
i = 0;
```

```
j = 0;
```

```
k = l;
```

```
while (i < n1 && j < n2) {
```

```
    if (L[i] <= R[j]) {
```

```
        arr[k] = L[i];
```

```
        i++;
```

```
    } else {
```

```
        arr[k] = R[j];
```

```
        j++;
```

```
    }
```

```
    k++;
```

```
}
```

```
while (i < n1) {
```

```
    arr[k] = L[i];
```

```
    i++;
```

```
    k++;
```

```
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++;  
    k++;  
}  
}
```

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = l + (r - l) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
        merge(arr, l, m, r);  
    }  
}
```

```
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);
```

```

int arr[n];

srand(time(NULL));

printf("Original random array:\n");

for (int i = 0; i < n; i++) {
    arr[i] = rand() % 100;
    printf("%d ", arr[i]);
}

printf("\n");

mergeSort(arr, 0, n - 1);

printf("Sorted array:\n");

printArray(arr, n);

return 0;
}

```

OR Q1) Write a C program to implement Static implementation of stack of integers with following operation: a. Initialize() b. Push() c. Pop() d. Isempty() e. Isfull() f. Peek().

C

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX_SIZE 100
```

```
int stack[MAX_SIZE];
```

```
int top = -1;
```

```
void Initialize() {
```

```
    top = -1;
    printf("Stack initialized.\n");
}
```

```
bool IsEmpty() {
    return top == -1;
}
```

```
bool IsFull() {
    return top == MAX_SIZE - 1;
}
```

```
void Push(int value) {
    if (IsFull()) {
        printf("Stack is full. Cannot push.\n");
    } else {
        stack[++top] = value;
        printf("%d pushed to stack.\n", value);
    }
}
```

```
int Pop() {
    if (IsEmpty()) {
        printf("Stack is empty. Cannot pop.\n");
        return -1;
    } else {
        return stack[top--];
    }
}
```

```
}
```

```
int Peek() {  
    if (IsEmpty()) {  
        printf("Stack is empty. No element to peek.\n");  
        return -1;  
    } else {  
        return stack[top];  
    }  
}
```

```
void Display() {  
    if (IsEmpty()) {  
        printf("Stack is empty.\n");  
        return;  
    }  
    printf("Stack elements: ");  
    for (int i = 0; i <= top; i++) {  
        printf("%d ", stack[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    Initialize();  
    Push(10);  
    Push(20);  
    Push(30);
```

```
printf("Peek element: %d\n", Peek());  
Display();  
printf("Popped element: %d\n", Pop());  
Display();  
return 0;  
}
```

Section II: Database Management Systems

Q2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Account (acctno, acct_type, balance, branch_name), Customer (custno, cust_name, cust_city)

-- Relationships: Customer-Account: One to Many.

-- Constraints: acct_type should be "saving" or "current" and balance is greater than zero.

```
CREATE TABLE Customer (  
    custno INT PRIMARY KEY,  
    cust_name VARCHAR(50),  
    cust_city VARCHAR(50)  
);
```

```
CREATE TABLE Account (  
    acctno INT PRIMARY KEY,  
    acct_type VARCHAR(10) CHECK (acct_type IN ('saving', 'current')),  
    balance DECIMAL(10, 2) CHECK (balance > 0),  
    branch_name VARCHAR(50),  
    custno INT,
```

```
FOREIGN KEY (custno) REFERENCES Customer(custno)
);
```

-- Insert records

```
INSERT INTO Customer VALUES (1, 'Sanjay', 'Pune');
```

```
INSERT INTO Customer VALUES (2, 'Ramnath', 'Mumbai');
```

```
INSERT INTO Customer VALUES (3, 'Anil', 'Pune');
```

```
INSERT INTO Account VALUES (101, 'saving', 15000.00, 'M.G.Road', 1);
```

```
INSERT INTO Account VALUES (102, 'current', 250000.00, 'M.G.Road', 2);
```

```
INSERT INTO Account VALUES (103, 'saving', 8000.00, 'Deccan', 3);
```

```
INSERT INTO Account VALUES (104, 'saving', 6000.00, 'M.G.Road', 1);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Display information of all saving accounts having balance greater than 5000.**

SQL

```
SELECT * FROM Account WHERE acct_type = 'saving' AND balance > 5000;
```

- **ii. Count customers in city "Pune".**

SQL

```
SELECT COUNT(*) FROM Customer WHERE cust_city = 'Pune';
```

- **iii. Find the total balance at branch "M.G.Road".**

SQL

```
SELECT SUM(balance) FROM Account WHERE branch_name = 'M.G.Road';
```

- **iv. Delete the record whose cust name is "Ramnath".**

SQL

```
DELETE FROM Account WHERE custno = (SELECT custno FROM Customer WHERE cust_name = 'Ramnath');
```

```
DELETE FROM Customer WHERE cust_name = 'Ramnath';
```

- **v. Change city of customer "Sanjay" to "Pune".**

SQL

```
UPDATE Customer SET cust_city = 'Pune' WHERE cust_name = 'Sanjay';
```

C. Write a query to find the number of account holders for each city.

SQL

```
SELECT cust_city, COUNT(DISTINCT C.custno)
```

```
FROM Customer C
```

```
JOIN Account A ON C.custno = A.custno
```

```
GROUP BY C.cust_city;
```

OR C. Create a view to print the account details along with customer and their branch details.

SQL

```
CREATE VIEW Full_Account_Details AS
```

```
SELECT A.acctno, A.acct_type, A.balance, A.branch_name, C.cust_name, C.cust_city
```

```
FROM Account A
```

```
JOIN Customer C ON A.custno = C.custno;
```

Slip 11

Section I: Data Structures

Q 1) Write a 'C' program to reverse a string using Static implementation of Stack.

C

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 100
```

```
char stack[MAX_SIZE];
```

```
int top = -1;
```

```
void push(char c) {  
    if (top < MAX_SIZE - 1) {  
        stack[++top] = c;  
    } else {  
        printf("Stack overflow!\n");  
    }  
}
```

```
char pop() {  
    if (top != -1) {  
        return stack[top--];  
    } else {  
        return '\0';  
    }  
}
```

```
int main() {  
    char str[MAX_SIZE];  
    printf("Enter a string: ");  
    scanf("%s", str);  
  
    for (int i = 0; i < strlen(str); i++) {  
        push(str[i]);  
    }  
  
    printf("Reversed string: ");  
    while (top != -1) {  
        printf("%c", pop());  
    }  
}
```

```
}  
  
printf("\n");  
  
return 0;  
  
}
```

OR Q 1) Read the data from the 'employee.txt' file and sort on age using Counting sort and write the sorted data to another file 'sortedemponage.txt'.

C

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>
```

```
typedef struct {  
    char name[50];  
    int age;  
    float salary;  
} Employee;
```

```
void countingSort(Employee arr[], int n) {  
    if (n <= 1) return;  
  
    int max_age = arr[0].age;  
    for (int i = 1; i < n; i++) {  
        if (arr[i].age > max_age) {  
            max_age = arr[i].age;  
        }  
    }  
}
```

```

Employee output[n];

int count[max_age + 1];

for (int i = 0; i <= max_age; i++) {
    count[i] = 0;
}

for (int i = 0; i < n; i++) {
    count[arr[i].age]++;
}

for (int i = 1; i <= max_age; i++) {
    count[i] += count[i - 1];
}

for (int i = n - 1; i >= 0; i--) {
    output[count[arr[i].age] - 1] = arr[i];
    count[arr[i].age]--;
}

for (int i = 0; i < n; i++) {
    arr[i] = output[i];
}

}

int main() {
    FILE *in_file, *out_file;

    Employee employees[100];

    int count = 0;

```

```

// Create a dummy employee.txt file
in_file = fopen("employee.txt", "w");
fprintf(in_file, "John 30 50000\n");
fprintf(in_file, "Jane 25 60000\n");
fprintf(in_file, "Peter 40 80000\n");
fprintf(in_file, "Alex 25 55000\n");
fclose(in_file);

in_file = fopen("employee.txt", "r");
if (in_file == NULL) {
    printf("Error opening input file.\n");
    return 1;
}

while (fscanf(in_file, "%s %d %f", employees[count].name, &employees[count].age,
&employees[count].salary) == 3) {
    count++;
}
fclose(in_file);

countingSort(employees, count);

out_file = fopen("sortedemponage.txt", "w");
if (out_file == NULL) {
    printf("Error opening output file.\n");
    return 1;
}

```

```

printf("Sorted data by age:\n");
for (int i = 0; i < count; i++) {
    printf("%s %d %.2f\n", employees[i].name, employees[i].age, employees[i].salary);
    fprintf(out_file, "%s %d %.2f\n", employees[i].name, employees[i].age,
employees[i].salary);
}
fclose(out_file);

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

```

-- Database Schema: Emp (eno, ename, salary), Project (pno, pname, budget)
-- Relationship: Emp-Project: Many to Many with descriptive attribute total_hours.

```

```

CREATE TABLE Emp (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),
    salary DECIMAL(10, 2)
);

```

```

CREATE TABLE Project (
    pno INT PRIMARY KEY,
    pname VARCHAR(50),

```

```
    budget DECIMAL(10, 2)
);
```

```
CREATE TABLE Emp_Project (
    eno INT,
    pno INT,
    total_hours INT,
    PRIMARY KEY (eno, pno),
    FOREIGN KEY (eno) REFERENCES Emp(eno),
    FOREIGN KEY (pno) REFERENCES Project(pno)
);
```

-- Insert records

```
INSERT INTO Emp VALUES (1, 'John Doe', 60000.00);
```

```
INSERT INTO Emp VALUES (2, 'Jane Smith', 75000.00);
```

```
INSERT INTO Emp VALUES (3, 'Peter Jones', 85000.00);
```

```
INSERT INTO Emp VALUES (4, 'Amit Gupta', 55000.00);
```

```
INSERT INTO Project VALUES (101, 'Web Development', 100000.00);
```

```
INSERT INTO Project VALUES (102, 'Mobile App', 150000.00);
```

```
INSERT INTO Project VALUES (103, 'Database Management', 80000.00);
```

```
INSERT INTO Emp_Project VALUES (1, 101, 120);
```

```
INSERT INTO Emp_Project VALUES (2, 101, 80);
```

```
INSERT INTO Emp_Project VALUES (2, 102, 150);
```

```
INSERT INTO Emp_Project VALUES (3, 103, 90);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Find the maximum budget.**

SQL

```
SELECT MAX(budget) FROM Project;
```

- **ii. Increase the salary of all employees by 10%.**

SQL

```
UPDATE Emp SET salary = salary * 1.10;
```

- **iii. Count the number of projects having duration greater than 100 hrs.**

SQL

```
SELECT COUNT(DISTINCT pno) FROM Emp_Project WHERE total_hours > 100;
```

- **iv. List all employees whose name ends with "a".**

SQL

```
SELECT * FROM Emp WHERE ename LIKE '%a';
```

- **v. Add column contact_number to employee table.**

SQL

```
ALTER TABLE Emp ADD COLUMN contact_number VARCHAR(15);
```

C. Write a query to list the names of employees who are not working on any project.

SQL

```
SELECT ename
```

```
FROM Emp
```

```
WHERE eno NOT IN (SELECT eno FROM Emp_Project);
```

OR C. Create a view to list the project details having maximum number of hours.

SQL

```
CREATE VIEW Project_Max_Hours AS
```

```
SELECT p.pname
```

```
FROM Project p
```

```
JOIN Emp_Project ep ON p.pno = ep.pno
```

```
ORDER BY ep.total_hours DESC
```

```
LIMIT 1;
```

Slip 12

Section I: Data Structures

Q 1) Perform the following operations using dynamic implementation of stack: i. Accept n integers from user and push it into the stack. ii. Reverses a string of characters using stack and check whether given string is palindrome or not.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
struct Node {  
    char data;  
    struct Node* next;  
};
```

```
void push(struct Node** top, char data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    if (newNode == NULL) return;  
    newNode->data = data;  
    newNode->next = *top;  
    *top = newNode;  
}
```

```
char pop(struct Node** top) {  
    if (*top == NULL) return '\0';  
    struct Node* temp = *top;  
    char data = temp->data;  
    *top = temp->next;  
    free(temp);
```

```
    return data;
}
```

```
int main() {
    int n, i;
    char str[100], reversed_str[100];
    struct Node* stack = NULL;

    // Part i: Push n integers
    printf("Enter the number of integers to push: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (i = 0; i < n; i++) {
        int item;
        scanf("%d", &item);
        push(&stack, item + '0'); // Pushing integers as chars for simplicity
    }

    // Part ii: Reverse a string and check for palindrome
    printf("\nEnter a string to check for palindrome: ");
    scanf("%s", str);

    stack = NULL;
    for (i = 0; i < strlen(str); i++) {
        push(&stack, str[i]);
    }

    i = 0;
```

```

while (stack != NULL) {
    reversed_str[i++] = pop(&stack);
}
reversed_str[i] = '\0';

printf("Original string: %s\n", str);
printf("Reversed string: %s\n", reversed_str);

if (strcmp(str, reversed_str) == 0) {
    printf("The string is a palindrome.\n");
} else {
    printf("The string is not a palindrome.\n");
}

return 0;
}

```

OR Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

C

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

```

```

void swap(int *xp, int *yp) {
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

```

```

void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        if (min_idx != i) {
            swap(&arr[min_idx], &arr[i]);
        }
    }
}

```

```

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

int main() {
    int n;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
}

```

```
srand(time(NULL));

printf("Original random array:\n");
for (int i = 0; i < n; i++) {
    arr[i] = rand() % 100;
    printf("%d ", arr[i]);
}
printf("\n");

selectionSort(arr, n);
printf("Sorted array:\n");
printArray(arr, n);

return 0;
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Sales_order(sorderno, s_order_date, order_amt), Client(clientno, name, address)

-- Relationship: Client-Sales_order: one-to-many

-- Constraints: order_amt should be greater than zero.

```
CREATE TABLE Client (
    clientno INT PRIMARY KEY,
    name VARCHAR(50),
```

```
address VARCHAR(100)
);
```

```
CREATE TABLE Sales_order (
    sorderno INT PRIMARY KEY,
    s_order_date DATE,
    order_amt DECIMAL(10, 2) CHECK (order_amt > 0),
    clientno INT,
    FOREIGN KEY (clientno) REFERENCES Client(clientno)
);
```

-- Insert records

```
INSERT INTO Client VALUES (1, 'Rahul Sharma', 'Pune');
INSERT INTO Client VALUES (2, 'Priya Singh', 'Nagpur');
INSERT INTO Client VALUES (3, 'Ajit Kulkarni', 'Kolhapur');
```

```
INSERT INTO Sales_order VALUES (101, '2024-01-10', 5000.00, 1);
INSERT INTO Sales_order VALUES (102, '2024-02-20', 12000.00, 1);
INSERT INTO Sales_order VALUES (103, '2024-03-05', 8000.00, 2);
INSERT INTO Sales_order VALUES (104, '2024-01-25', 15000.00, 1);
INSERT INTO Sales_order VALUES (105, '2024-02-22', 9000.00, 3);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Update the client address of all clients from "Nagpur" to "Kolhapur".**

SQL

```
UPDATE Client SET address = 'Kolhapur' WHERE address = 'Nagpur';
```

- **ii. Delete all sales order whose client number is 30.** (Assuming a typo, and it should be a client number that exists, e.g., 2).

SQL

DELETE FROM Sales_order WHERE clientno = 2;

- **iii. Display all sale records having order date before "23/02/2024".**

SQL

SELECT * FROM Sales_order WHERE s_order_date < '2024-02-23';

- **iv. Display sales order having maximum order amount.**

SQL

SELECT * FROM Sales_order ORDER BY order_amt DESC LIMIT 1;

- **v. Add column order_status to the Sales_order table.**

SQL

ALTER TABLE Sales_order ADD COLUMN order_status VARCHAR(20);

C. Write a query to Display client details having maximum sales orders.

SQL

SELECT C.clientno, C.name, C.address

FROM Client C

JOIN Sales_order S ON C.clientno = S.clientno

GROUP BY C.clientno, C.name, C.address

ORDER BY COUNT(S.sorderno) DESC

LIMIT 1;

OR C. Create a view to list names of clients having more than 2 sales orders.

SQL

CREATE VIEW Clients_With_More_Than_Two_Orders AS

SELECT C.name

FROM Client C

JOIN Sales_order S ON C.clientno = S.clientno

GROUP BY C.clientno, C.name

HAVING COUNT(S.sorderno) > 2;

Section I: Data Structures

Q 1) Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

C

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int x) {
```

```
    int low = 0;
```

```
    int high = n - 1;
```

```
    while (low <= high) {
```

```
        int mid = low + (high - low) / 2;
```

```
        if (arr[mid] == x) {
```

```
            return mid;
```

```
        }
```

```
        if (arr[mid] < x) {
```

```
            low = mid + 1;
```

```
        } else {
```

```
            high = mid - 1;
```

```
        }
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int n, x, result;
```

```
    printf("Enter the number of sorted elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```

printf("Enter %d sorted integers:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

printf("Enter the value to search: ");
scanf("%d", &x);

result = binarySearch(arr, n, x);

if (result != -1) {
    printf("Number %d is present at position %d.\n", x, result + 1);
} else {
    printf("Number %d is not present in the array.\n", x);
}

return 0;
}

```

OR Q 1) Perform the following operations using dynamic implementation of Queue:
i. Insert n elements in a queue. ii. Delete the element from the queue and display it.

C

```

#include <stdio.h>

#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
}

```

```
};
```

```
struct Node *front = NULL, *rear = NULL;
```

```
void insert(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) return;
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    if (rear == NULL) {
```

```
        front = rear = newNode;
```

```
    } else {
```

```
        rear->next = newNode;
```

```
        rear = newNode;
```

```
    }
```

```
}
```

```
void delete() {
```

```
    if (front == NULL) {
```

```
        printf("Queue is empty.\n");
```

```
        return;
```

```
    }
```

```
    struct Node* temp = front;
```

```
    front = front->next;
```

```
    if (front == NULL) {
```

```
        rear = NULL;
```

```
    }
```

```
    printf("Deleted element: %d\n", temp->data);  
    free(temp);  
}
```

```
void display() {  
    if (front == NULL) {  
        printf("Queue is empty.\n");  
        return;  
    }  
    printf("Queue elements: ");  
    struct Node* temp = front;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
    printf("\n");  
}
```

```
int main() {  
    int n, i, element;  
    printf("Enter the number of elements to insert: ");  
    scanf("%d", &n);  
    printf("Enter %d elements:\n", n);  
    for (i = 0; i < n; i++) {  
        scanf("%d", &element);  
        insert(element);  
    }  
}
```

```
display();  
delete();  
display();  
  
return 0;  
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Bus (Busno, capacity, depot_name), Route (Routeno, source, destination,no_of_stations)

-- Relationship: Bus-Route: Many to one

-- Constraint: Bus capacity should not be null.

```
CREATE TABLE Route (  
    Routeno INT PRIMARY KEY,  
    source VARCHAR(50),  
    destination VARCHAR(50),  
    no_of_stations INT  
);
```

```
CREATE TABLE Bus (  
    Busno INT PRIMARY KEY,  
    capacity INT NOT NULL,  
    depot_name VARCHAR(50),  
    Routeno INT,
```

```
FOREIGN KEY (Routeno) REFERENCES Route(Routeno)
);
```

-- Insert records

```
INSERT INTO Route VALUES (41, 'Pune station', 'Swargate', 12);
```

```
INSERT INTO Route VALUES (2, 'Pimpri', 'Hadapsar', 15);
```

```
INSERT INTO Route VALUES (10, 'Swargate', 'Kothrud', 8);
```

```
INSERT INTO Bus VALUES (101, 50, 'Swargate', 41);
```

```
INSERT INTO Bus VALUES (102, 45, 'Hadapsar', 2);
```

```
INSERT INTO Bus VALUES (204, 60, 'Swargate', 10);
```

```
INSERT INTO Bus VALUES (103, 55, 'Pimpri', 2);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. List all buses at depot "Swargate".**

SQL

```
SELECT * FROM Bus WHERE depot_name = 'Swargate';
```

- **ii. Delete Bus whose Bus number is "204".**

SQL

```
DELETE FROM Bus WHERE Busno = 204;
```

- **iii. List all buses on route number 41.**

SQL

```
SELECT * FROM Bus WHERE Routeno = 41;
```

- **iv. List the route details having number of stations greater than 10.**

SQL

```
SELECT * FROM Route WHERE no_of_stations > 10;
```

- **v. List all routes starting from "Pune station".**

SQL

```
SELECT * FROM Route WHERE source = 'Pune station';
```

C. Write a query to delete all the buses on routes in which total stations are less than 3.

SQL

```
DELETE FROM Bus WHERE Routeno IN (SELECT Routeno FROM Route WHERE  
no_of_stations < 3);
```

OR C. Create a view to list all buses and route details from "Pimpri" to "Hadapsar".

SQL

```
CREATE VIEW Pimpri_Hadapsar_Buses AS
```

```
SELECT B.Busno, B.capacity, B.depot_name, R.Routeno, R.source, R.destination,  
R.no_of_stations
```

```
FROM Bus B
```

```
JOIN Route R ON B.Routeno = R.Routeno
```

```
WHERE R.source = 'Pimpri' AND R.destination = 'Hadapsar';
```

Slip 14

Section I: Data Structures

Q 1) Read the data from "Student.txt (Stud_name, Age, Percentage)" file and sort on Percentage using selection sort. (Create a separate sorted file).

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
    char name[50];
```

```
    int age;
```

```
    float percentage;
```

```
} Student;
```

```
void swap(Student* a, Student* b) {  
  
    Student temp = *a;  
  
    *a = *b;  
  
    *b = temp;  
  
}
```

```
void selectionSort(Student arr[], int n) {  
  
    int i, j, min_idx;  
  
    for (i = 0; i < n - 1; i++) {  
  
        min_idx = i;  
  
        for (j = i + 1; j < n; j++) {  
  
            if (arr[j].percentage < arr[min_idx].percentage) {  
  
                min_idx = j;  
  
            }  
  
        }  
  
        if (min_idx != i) {  
  
            swap(&arr[min_idx], &arr[i]);  
  
        }  
  
    }  
  
}
```

```
int main() {  
  
    FILE *in_file, *out_file;  
  
    Student students[100];  
  
    int count = 0;  
  
  
    // Create a dummy input file  
  
    in_file = fopen("Student.txt", "w");
```

```
fprintf(in_file, "John 22 85.5\n");  
fprintf(in_file, "Jane 20 90.0\n");  
fprintf(in_file, "Peter 25 78.9\n");  
fprintf(in_file, "Alex 21 92.1\n");  
fclose(in_file);
```

```
in_file = fopen("Student.txt", "r");  
if (in_file == NULL) {  
    printf("Error opening input file.\n");  
    return 1;  
}
```

```
while (fscanf(in_file, "%s %d %f", students[count].name, &students[count].age,  
&students[count].percentage) == 3) {  
    count++;  
}  
fclose(in_file);
```

```
selectionSort(students, count);
```

```
out_file = fopen("sorted_by_percentage.txt", "w");  
if (out_file == NULL) {  
    printf("Error opening output file.\n");  
    return 1;  
}
```

```
printf("Sorted data by percentage:\n");  
for (int i = 0; i < count; i++) {
```

```

        printf("%s %d %.2f\n", students[i].name, students[i].age, students[i].percentage);

        fprintf(out_file, "%s %d %.2f\n", students[i].name, students[i].age,
students[i].percentage);

    }

    fclose(out_file);

    return 0;
}

```

OR Q 1) Write a menu driven program for the following operations on singly linked list: i. Create a linked list and display it. ii. Accept an element from user and concatenate it with a created link list.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node* next;
};

```

```

struct Node* createList() {
    struct Node* head = NULL;
    struct Node* newNode, *temp;
    int data, n;

```

```

    printf("Enter the number of nodes to create: ");
    scanf("%d", &n);

```

```

    if (n <= 0) return NULL;

```

```

printf("Enter %d data elements:\n", n);
for (int i = 0; i < n; i++) {
    newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) break;
    printf("Enter element %d: ", i + 1);
    scanf("%d", &data);
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
return head;
}

```

```

void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
}

```

```

printf("Linked list elements: ");
struct Node* temp = head;
while (temp != NULL) {
    printf("%d -> ", temp->data);
    temp = temp->next;
}
printf("NULL\n");
}

```

```

void concatenate(struct Node** head) {
    if (*head == NULL) {
        *head = createList();
        return;
    }
    struct Node *temp = *head;
    while(temp->next != NULL) {
        temp = temp->next;
    }
    printf("Enter the element to concatenate: ");
    int data;
    scanf("%d", &data);
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) return;
    newNode->data = data;
    newNode->next = NULL;
    temp->next = newNode;
}

```

```
int main() {  
  
    struct Node* head = NULL;  
  
    int choice;  
  
    do {  
  
        printf("\n--- Singly Linked List Operations ---\n");  
  
        printf("1. Create and Display\n");  
  
        printf("2. Concatenate an element\n");  
  
        printf("3. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
  
        switch (choice) {  
  
            case 1:  
  
                head = createList();  
  
                displayList(head);  
  
                break;  
  
            case 2:  
  
                concatenate(&head);  
  
                displayList(head);  
  
                break;  
  
            case 3:  
  
                printf("Exiting program.\n");  
  
                break;  
  
            default:  
  
                printf("Invalid choice. Please try again.\n");  
  
        }  
  
    } while (choice != 3);  
}
```

```
    return 0;
}
```

Slip 15

Section I: Data Structures

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void insertionSort(int arr[], int n) {
```

```
    int i, key, j;
```

```
    for (i = 1; i < n; i++) {
```

```
        key = arr[i];
```

```
        j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j + 1] = arr[j];
```

```
            j = j - 1;
```

```
        }
```

```
        arr[j + 1] = key;
```

```
    }
```

```
}
```

```
void printArray(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++) {  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
}
```

```
int main() {  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
  
    int arr[n];  
    srand(time(0));  
    printf("Original array (random elements):\n");  
    for (int i = 0; i < n; i++) {  
        arr[i] = rand() % 100; // Generate random numbers up to 99  
        printf("%d ", arr[i]);  
    }  
    printf("\n");  
  
    insertionSort(arr, n);  
  
    printf("Sorted array:\n");  
    printArray(arr, n);  
  
    return 0;  
}
```

OR Q 1) Perform the following operations using dynamic implementation of Queue:
i. Insert n elements in a queue. ii. Delete the element from the queue and display it.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node *front = NULL, *rear = NULL;
```

```
void insert(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    if (newNode == NULL) return;  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (rear == NULL) {  
        front = rear = newNode;  
    } else {  
        rear->next = newNode;  
        rear = newNode;  
    }  
}
```

```
void delete() {
```

```
if (front == NULL) {  
    printf("Queue is empty.\n");  
    return;  
}  
  
struct Node* temp = front;  
front = front->next;  
  
if (front == NULL) {  
    rear = NULL;  
}  
  
printf("Deleted element: %d\n", temp->data);  
free(temp);  
}
```

```
void display() {  
    if (front == NULL) {  
        printf("Queue is empty.\n");  
        return;  
    }  
  
    printf("Queue elements: ");  
    struct Node* temp = front;  
    while (temp != NULL) {  
        printf("%d ", temp->data);  
        temp = temp->next;  
    }  
  
    printf("\n");  
}
```

```
int main() {
```

```

int n, i, element;

printf("Enter the number of elements to insert: ");

scanf("%d", &n);

printf("Enter %d elements:\n", n);

for (i = 0; i < n; i++) {
    scanf("%d", &element);

    insert(element);
}

display();

delete();

display();

return 0;
}

```

Section II: Database Management Systems

Q2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Game (gname, noofplayers, coachname, captain_name), Player (pno, pname)

-- Relationship: Game-Player: one-to-many.

-- Constraint: captain_name should be uppercase.

```

CREATE TABLE Game (
    gname VARCHAR(50) PRIMARY KEY,
    noofplayers INT,

```

```
coachname VARCHAR(50),  
captain_name VARCHAR(50) CHECK (captain_name = UPPER(captain_name))  
);
```

```
CREATE TABLE Player (  
    pno INT PRIMARY KEY,  
    pname VARCHAR(50),  
    gname VARCHAR(50),  
    FOREIGN KEY (gname) REFERENCES Game(gname)  
);
```

-- Insert records

```
INSERT INTO Game VALUES ('Cricket', 11, 'Tendulkar', 'VIRAT');  
INSERT INTO Game VALUES ('Hockey', 11, 'Dhoni', 'MANPREET');  
INSERT INTO Game VALUES ('Kho kho', 9, 'Rahul Dravid', 'VIKAS');
```

```
INSERT INTO Player VALUES (1, 'Rohit Sharma', 'Cricket');  
INSERT INTO Player VALUES (2, 'M.S. Dhoni', 'Cricket');  
INSERT INTO Player VALUES (3, 'Manpreet Singh', 'Hockey');  
INSERT INTO Player VALUES (4, 'Pawan Sehrawat', 'Kho kho');  
INSERT INTO Player VALUES (5, 'Hardik Pandya', 'Cricket');
```

B. Execute the following queries in PostgreSQL (any 3).

- **i. List the names of players playing "hockey".**

SQL

```
SELECT pname FROM Player WHERE gname = 'Hockey';
```

- **ii. Give the average number of players.**

SQL

```
SELECT AVG(noofplayers) FROM Game;
```

- **iii. Delete the records of players playing 'kho kho'.**

SQL

```
DELETE FROM Player WHERE gname = 'Kho kho';
```

- **iv. List names of players not playing "cricket".**

SQL

```
SELECT pname FROM Player WHERE gname != 'Cricket';
```

- **v. Update the coach name from "Dhoni" to "Tendulkar" for game "Cricket".**

SQL

```
UPDATE Game SET coachname = 'Tendulkar' WHERE gname = 'Cricket';
```

C. Write a query to display the game details with maximum number of players.

SQL

```
SELECT *
```

```
FROM Game
```

```
ORDER BY noofplayers DESC
```

```
LIMIT 1;
```

OR C. Create a view to print game-wise list of Players along with their game name.

SQL

```
CREATE VIEW Game_Wise_Players AS
```

```
SELECT G.gname, P.pname
```

```
FROM Game G
```

```
JOIN Player P ON G.gname = P.gname
```

```
ORDER BY G.gname, P.pname;
```

Slip 16

Section I: Data Structures

Q1) Write a menu driven program for the following operations: i. Reverse a linked list and display it. ii. Accept an element from user and Concatenate it with a created link list. iii. Merge two linked list and display it.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* createList(int n) {
```

```
    struct Node* head = NULL;
```

```
    struct Node* newNode, *temp;
```

```
    int data;
```

```
    if (n <= 0) return NULL;
```

```
    printf("Enter %d data elements:\n", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
        if (newNode == NULL) break;
```

```
        printf("Enter element %d: ", i + 1);
```

```
        scanf("%d", &data);
```

```
        newNode->data = data;
```

```
        newNode->next = NULL;
```

```
        if (head == NULL) {
```

```
            head = newNode;
```

```
        } else {
```

```
            temp = head;
```

```
            while (temp->next != NULL) {
```

```

        temp = temp->next;
    }
    temp->next = newNode;
}
}
return head;
}

```

```

void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked list elements: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

void reverseList(struct Node** head_ref) {
    struct Node* prev = NULL;
    struct Node* current = *head_ref;
    struct Node* next = NULL;
    while (current != NULL) {
        next = current->next;

```

```

        current->next = prev;

        prev = current;

        current = next;
    }

    *head_ref = prev;

    printf("List reversed.\n");
}

```

```

void concatenateElement(struct Node** head) {
    if (*head == NULL) {
        *head = createList(1);

        return;
    }

    struct Node *temp = *head;
    while(temp->next != NULL) {
        temp = temp->next;
    }

    printf("Enter the element to concatenate: ");

    int data;

    scanf("%d", &data);

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) return;

    newNode->data = data;

    newNode->next = NULL;

    temp->next = newNode;

    printf("Element concatenated.\n");
}

```

```

void mergeLists(struct Node** head1, struct Node** head2) {
    if (*head1 == NULL) {
        *head1 = *head2;
    } else if (*head2 != NULL) {
        struct Node* temp = *head1;
        while(temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = *head2;
    }
    printf("Lists merged.\n");
}

```

```

int main() {
    struct Node *head1 = NULL, *head2 = NULL;
    int choice;

    do {
        printf("\n--- Singly Linked List Operations ---\n");
        printf("1. Reverse and Display a list\n");
        printf("2. Concatenate an element to a list\n");
        printf("3. Merge two lists and display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:

```

```
printf("Creating a list to reverse...\n");
```

```
head1 = createList(3);
```

```
printf("Original list:\n");
```

```
displayList(head1);
```

```
reverseList(&head1);
```

```
printf("Reversed list:\n");
```

```
displayList(head1);
```

```
break;
```

case 2:

```
printf("Creating a list...\n");
```

```
head1 = createList(3);
```

```
concatenateElement(&head1);
```

```
displayList(head1);
```

```
break;
```

case 3:

```
printf("Creating list 1...\n");
```

```
head1 = createList(2);
```

```
printf("Creating list 2...\n");
```

```
head2 = createList(3);
```

```
mergeLists(&head1, &head2);
```

```
printf("Merged list:\n");
```

```
displayList(head1);
```

```
break;
```

case 4:

```
printf("Exiting program.\n");
```

```
break;
```

default:

```
printf("Invalid choice.\n");
```

```

    }
} while (choice != 4);

return 0;
}

```

OR Q1) Read the data from file 'cities.txt' containing names of 10 cities and their STD codes. Accept a name of the city from user and use linear search algorithm to check whether the name is present in the file and output the STD code, otherwise output "city not in the list".

C

```

#include <stdio.h>

#include <string.h>

#include <stdlib.h>

int main() {
    FILE *file;

    char searchCity[50], cityName[50], stdCode[10];

    int found = 0;

    // Creating a dummy cities.txt file

    file = fopen("cities.txt", "w");

    fprintf(file, "Pune 020\n");

    fprintf(file, "Mumbai 022\n");

    fprintf(file, "Delhi 011\n");

    fprintf(file, "Nashik 0253\n");

    fprintf(file, "Chennai 044\n");

    fclose(file);

    file = fopen("cities.txt", "r");

```

```

if (file == NULL) {
    printf("Error opening file.\n");
    return 1;
}

printf("Enter the city name to search: ");
scanf("%s", searchCity);

while (fscanf(file, "%s %s", cityName, stdCode) != EOF) {
    if (strcmp(cityName, searchCity) == 0) {
        printf("City found. STD code: %s\n", stdCode);
        found = 1;
        break;
    }
}

if (!found) {
    printf("City not in the list.\n");
}

fclose(file);
return 0;
}

```

Section II: Database Management Systems

Q2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Student (sno,s_name, s_class), Teacher (tno,t_name, yrs_experience)

-- Relationship: Student-Teacher: M-M with descriptive attribute Subject.

-- Constraint: s_class should be either "FY", "SY" or "TY"

```
CREATE TABLE Student (  
    sno INT PRIMARY KEY,  
    s_name VARCHAR(50),  
    s_class VARCHAR(10) CHECK (s_class IN ('FY', 'SY', 'TY'))  
);
```

```
CREATE TABLE Teacher (  
    tno INT PRIMARY KEY,  
    t_name VARCHAR(50),  
    yrs_experience INT  
);
```

```
CREATE TABLE Student_Teacher (  
    sno INT,  
    tno INT,  
    subject VARCHAR(50),  
    PRIMARY KEY (sno, tno, subject),  
    FOREIGN KEY (sno) REFERENCES Student(sno),  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

-- Insert records

```
INSERT INTO Student VALUES (101, 'Rahul', 'FY');
```

```
INSERT INTO Student VALUES (102, 'Priya', 'SY');
INSERT INTO Student VALUES (103, 'Sanjay', 'TY');
INSERT INTO Student VALUES (104, 'Nikhil', 'SY');
INSERT INTO Student VALUES (105, 'Sagar', 'TY');
```

```
INSERT INTO Teacher VALUES (1, 'Dr. Wani', 10);
INSERT INTO Teacher VALUES (2, 'Mrs. Deshmukh', 5);
INSERT INTO Teacher VALUES (3, 'Mr. Jadhav', 15);
INSERT INTO Teacher VALUES (4, 'Mr. Patil', 8);
```

```
INSERT INTO Student_Teacher VALUES (101, 1, 'Data Structures');
INSERT INTO Student_Teacher VALUES (102, 2, 'Maths');
INSERT INTO Student_Teacher VALUES (103, 3, 'Maths');
INSERT INTO Student_Teacher VALUES (104, 1, 'Maths');
INSERT INTO Student_Teacher VALUES (105, 4, 'DBMS');
INSERT INTO Student_Teacher VALUES (101, 2, 'DBMS');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. List teachers who has more than 5 years of experience.**

SQL

```
SELECT * FROM Teacher WHERE yrs_experience > 5;
```

- **ii. Count all students who have taken subject "Data Structure".**

SQL

```
SELECT COUNT(DISTINCT sno) FROM Student_Teacher WHERE subject = 'Data Structure';
```

- **iii. List the names of all students from SY Class.**

SQL

```
SELECT s_name FROM Student WHERE s_class = 'SY';
```

- **iv. Change experience of teacher named "Pawar" to 20.**

SQL

```
UPDATE Teacher SET yrs_experience = 20 WHERE t_name = 'Pawar';
```

- **v. Display classwise details of students.**

SQL

```
SELECT s_class, COUNT(*) FROM Student GROUP BY s_class;
```

C. Write a query to list the names of all teachers with their subjects along with the student names to whom they are teaching.

SQL

```
SELECT T.t_name, ST.subject, S.s_name
FROM Teacher T
JOIN Student_Teacher ST ON T.tno = ST.tno
JOIN Student S ON ST.sno = S.sno
ORDER BY T.t_name, ST.subject, S.s_name;
```

OR C. Create a view to find the teacher teaching maximum number of subjects.

SQL

```
CREATE VIEW Teacher_Max_Subjects AS
SELECT t.t_name
FROM Teacher t
JOIN Student_Teacher st ON t.tno = st.tno
GROUP BY t.t_name
ORDER BY COUNT(st.subject) DESC
LIMIT 1;
```

Slip 17

Section I: Data Structures

Q 1) Sort a random array of n integers (accept the value of n from user) in ascending order by using recursive Quick sort algorithm.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void swap(int* a, int* b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j] <= pivot) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```
        }
```

```
    }
```

```
    swap(&arr[i + 1], &arr[high]);
```

```
    return (i + 1);
```

```
}
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

```
void printArray(int arr[], int n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    srand(time(NULL));
```

```
    printf("Original array (random elements):\n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        arr[i] = rand() % 100;
```

```
        printf("%d ", arr[i]);
```

```
    }
```

```
    printf("\n");
```

```
    quickSort(arr, 0, n - 1);
```

```
    printf("Sorted array using recursive quick sort:\n");
```

```
    printArray(arr, n);
```

```
    return 0;
```

```
}
```

OR Q 1) Perform the following operations using implementation of stack (Static or dynamic): i. Accept n integers from user and push it into the stack. ii. Reverses a string of characters.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
// Static stack implementation
```

```
#define MAX_SIZE 100
```

```
char static_stack[MAX_SIZE];
```

```
int static_top = -1;
```

```
void static_push(char data) {
```

```
    if (static_top < MAX_SIZE - 1) {
```

```
        static_stack[++static_top] = data;
```

```
    } else {
```

```
        printf("Static stack overflow.\n");
```

```
    }
```

```
}
```

```
char static_pop() {
```

```
    if (static_top == -1) {
```

```
        printf("Static stack underflow.\n");
```

```
        return '\0';
```

```
    }
```

```
    return static_stack[static_top--];
```

```
}
```

```
// Dynamic stack implementation
```

```
struct Node {
```

```
    char data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* dynamic_top = NULL;
```

```
void dynamic_push(char data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) return;
```

```
    newNode->data = data;
```

```
    newNode->next = dynamic_top;
```

```
    dynamic_top = newNode;
```

```
}
```

```
char dynamic_pop() {
```

```
    if (dynamic_top == NULL) return '\0';
```

```
    struct Node* temp = dynamic_top;
```

```
    char data = temp->data;
```

```
    dynamic_top = temp->next;
```

```
    free(temp);
```

```
    return data;
```

```
}
```

```
int main() {
```

```
    int choice;
```

```
printf("Choose stack implementation (1 for static, 2 for dynamic): ");
scanf("%d", &choice);

if (choice == 1) {
    // Static implementation

    int n, item;

    printf("Enter the number of integers to push: ");
    scanf("%d", &n);
    printf("Enter %d integers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &item);
        static_push(item + '0'); // Convert int to char
    }

    char str[100];
    printf("\nEnter a string to reverse: ");
    scanf("%s", str);
    static_top = -1; // Reset stack for string
    for (int i = 0; i < strlen(str); i++) {
        static_push(str[i]);
    }

    printf("Reversed string: ");
    while (static_top != -1) {
        printf("%c", static_pop());
    }
    printf("\n");
}
```

```

} else if (choice == 2) {

    // Dynamic implementation

    int n, item;

    printf("Enter the number of integers to push: ");

    scanf("%d", &n);

    printf("Enter %d integers:\n", n);

    for (int i = 0; i < n; i++) {

        scanf("%d", &item);

        dynamic_push(item + '0');

    }


    char str[100];

    printf("\nEnter a string to reverse: ");

    scanf("%s", str);

    dynamic_top = NULL; // Reset stack

    for (int i = 0; i < strlen(str); i++) {

        dynamic_push(str[i]);

    }


    printf("Reversed string: ");

    while (dynamic_top != NULL) {

        printf("%c", dynamic_pop());

    }

    printf("\n");

} else {

    printf("Invalid choice.\n");

}

```

```
    return 0;
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Property (Pno, area_Sqft, location, city), Owner (oname, address, phone)

-- Relationship: Property - Owner are related with Many to One relationship.

```
CREATE TABLE Owner (
    oname VARCHAR(50) PRIMARY KEY,
    address VARCHAR(100),
    phone VARCHAR(15)
);

CREATE TABLE Property (
    Pno INT PRIMARY KEY,
    area_Sqft INT,
    location VARCHAR(50),
    city VARCHAR(50),
    oname VARCHAR(50),
    FOREIGN KEY (oname) REFERENCES Owner(oname)
);
```

-- Insert records

```
INSERT INTO Owner VALUES ('Mr. Rahane', 'Pune', '9876543210');
```

```
INSERT INTO Owner VALUES ('Mr. Baravkar', 'Mumbai', '9988776655');
```

```
INSERT INTO Owner VALUES ('Ms. Sharma', 'Nashik', '9012345678');
```

```
INSERT INTO Property VALUES (101, 2500, 'Kothrud', 'Pune', 'Mr. Rahane');
```

```
INSERT INTO Property VALUES (102, 1500, 'Deccan', 'Pune', 'Ms. Sharma');
```

```
INSERT INTO Property VALUES (103, 12000, 'Kothrud', 'Pune', 'Mr. Baravkar');
```

```
INSERT INTO Property VALUES (104, 3000, 'Shivajinagar', 'Nashik', 'Ms. Sharma');
```

```
INSERT INTO Property VALUES (105, 1000, 'Kothrud', 'Pune', 'Mr. Rahane');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Give the list of all properties in Nashik city.**

SQL

```
SELECT * FROM Property WHERE city = 'Nashik';
```

- **ii. Give the names of owners whose property is greater than 2000sq feet.**

SQL

```
SELECT DISTINCT oname FROM Property WHERE area_Sqft > 2000;
```

- **iii. Find the number of properties owned by "Mr. Rahane".**

SQL

```
SELECT COUNT(*) FROM Property WHERE oname = 'Mr. Rahane';
```

- **iv. Count all properties of "Pune" city.**

SQL

```
SELECT COUNT(*) FROM Property WHERE city = 'Pune';
```

- **v. Delete all properties in "Pune" city owned by "Mr.Baravkar".**

SQL

```
DELETE FROM Property WHERE city = 'Pune' AND oname = 'Mr. Baravkar';
```

C. Write a query to find the names of all owners who own a property in Pune city, located in 'Kothrud' area and which has an area greater than 10000sqft.

SQL

```
SELECT oname
FROM Property
WHERE city = 'Pune' AND location = 'Kothrud' AND area_Sqft > 10000;
```

OR C. Create a view to find the owner wise list of properties.

```
SQL
CREATE VIEW Owner_Wise_Properties AS
SELECT o.ename, p.Pno, p.area_Sqft, p.location, p.city
FROM Owner o
JOIN Property p ON o.ename = p.ename
ORDER BY o.ename;
```

Slip 18

Section I: Data Structures

Q1) Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void swap(int *xp, int *yp) {
```

```
    int temp = *xp;
```

```
    *xp = *yp;
```

```
    *yp = temp;
```

```
}
```

```
void bubbleSort(int arr[], int n) {
```

```
    int i, j;
```

```

for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            swap(&arr[j], &arr[j + 1]);
        }
    }
}
}

```

```

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

```

```

int main() {
    int n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);
    int arr[n];
    srand(time(NULL));

    printf("Original random array:\n");
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
}

```

```

printf("\n");

bubbleSort(arr, n);

printf("Sorted array:\n");

printArray(arr, n);

return 0;
}

```

OR Q1) Write a program to perform the following operation on stack. Convert an infix expression of the form $(a*(b-c*d)/((a+d)/b))$ into its equivalent postfix notation.

C

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>

#define MAX_SIZE 100

char stack[MAX_SIZE];

int top = -1;

void push(char c) {
    if (top >= MAX_SIZE - 1) return;
    stack[++top] = c;
}

char pop() {
    if (top == -1) return '\0';
    return stack[top--];
}

```

```
}
```

```
int precedence(char op) {  
    if (op == '+' || op == '-') return 1;  
    if (op == '*' || op == '/') return 2;  
    return 0;  
}
```

```
void infixToPostfix(char infix[]) {  
    char postfix[MAX_SIZE];  
    int j = 0;  
  
    for (int i = 0; infix[i] != '\0'; i++) {  
        char ch = infix[i];  
  
        if (isalnum(ch)) {  
            postfix[j++] = ch;  
        } else if (ch == '(') {  
            push(ch);  
        } else if (ch == ')') {  
            while (top != -1 && stack[top] != '(') {  
                postfix[j++] = pop();  
            }  
            if (top != -1) pop(); // Pop the '('  
        } else {  
            while (top != -1 && precedence(stack[top]) >= precedence(ch)) {  
                postfix[j++] = pop();  
            }  
        }  
    }  
}
```

```

        push(ch);
    }
}

while (top != -1) {
    postfix[j++] = pop();
}

postfix[j] = '\0';

printf("Postfix expression: %s\n", postfix);
}

int main() {
    char infix[100] = "(a*(b-c*d)/((a+d)/b))";
    printf("Infix expression: %s\n", infix);
    infixToPostfix(infix);
    return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Student (rno,name,city), Teacher(tno,tname,phone_no,salary)

-- Relationship: Student-Teacher: Many-to-Many with subject as a descriptive attribute.

CREATE TABLE Student (

 rno INT PRIMARY KEY,

```
name VARCHAR(50),  
city VARCHAR(50)  
);
```

```
CREATE TABLE Teacher (  
    tno INT PRIMARY KEY,  
    tname VARCHAR(50),  
    phone_no VARCHAR(15),  
    salary DECIMAL(10, 2)  
);
```

```
CREATE TABLE Student_Teacher (  
    rno INT,  
    tno INT,  
    subject VARCHAR(50),  
    PRIMARY KEY (rno, tno, subject),  
    FOREIGN KEY (rno) REFERENCES Student(rno),  
    FOREIGN KEY (tno) REFERENCES Teacher(tno)  
);
```

-- Insert records

```
INSERT INTO Student VALUES (1, 'Rajesh', 'Pune');  
INSERT INTO Student VALUES (2, 'Priya', 'Sangamner');  
INSERT INTO Student VALUES (3, 'Sanjay', 'Pune');  
INSERT INTO Student VALUES (4, 'Amit', 'Sangamner');
```

```
INSERT INTO Teacher VALUES (101, 'Dr. Wani', '9834233234', 60000.00);  
INSERT INTO Teacher VALUES (102, 'Dr. Mulay', '9834233235', 75000.00);
```

```
INSERT INTO Teacher VALUES (103, 'Prof. Jadhav', '9000011111', 50000.00);
```

```
INSERT INTO Student_Teacher VALUES (1, 101, 'DBMS');
```

```
INSERT INTO Student_Teacher VALUES (2, 101, 'IKS');
```

```
INSERT INTO Student_Teacher VALUES (3, 102, 'DBMS');
```

```
INSERT INTO Student_Teacher VALUES (4, 103, 'Electronics');
```

```
INSERT INTO Student_Teacher VALUES (2, 103, 'Electronics');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. List the students from "Sangamner"city.**

SQL

```
SELECT * FROM Student WHERE city = 'Sangamner';
```

- **ii. Find the maximum salary of teachers.**

SQL

```
SELECT MAX(salary) FROM Teacher;
```

- **iii. Change the phone number of teacher 'Dr. Mulay' to 9834233235.**

SQL

```
UPDATE Teacher SET phone_no = '9834233235' WHERE tname = 'Dr. Mulay';
```

- **iv. List the details of the teachers in the sorted order of their name.**

SQL

```
SELECT * FROM Teacher ORDER BY tname;
```

- **v. Count the number of students who have taken subject "IKS".**

SQL

```
SELECT COUNT(DISTINCT rno) FROM Student_Teacher WHERE subject = 'IKS';
```

C. Write a query to list the names of all students who are taught by teacher "Dr.Wani".

SQL

```
SELECT S.name
```

```
FROM Student S
```

```
JOIN Student_Teacher ST ON S.rno = ST.rno
```

```
JOIN Teacher T ON ST.tno = T.tno
```

```
WHERE T.tname = 'Dr. Wani';
```

OR C. Create a view to list the details of students who have not taken 'Electronics' subject.

SQL

```
CREATE VIEW Students_No_Electronics AS
```

```
SELECT s.rno, s.name, s.city
```

```
FROM Student s
```

```
WHERE s.rno NOT IN (
```

```
    SELECT rno
```

```
    FROM Student_Teacher
```

```
    WHERE subject = 'Electronics'
```

```
);
```

Slip 19

Section I: Data Structures

Q1) Read the data from the file "employee.txt" and sort on age using Bubble sort.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
    char name[50];
```

```
    int age;
```

```
    float salary;
```

```
} Employee;
```

```
void swap(Employee* a, Employee* b) {  
    Employee temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void bubbleSort(Employee arr[], int n) {  
    int i, j;  
    for (i = 0; i < n - 1; i++) {  
        for (j = 0; j < n - i - 1; j++) {  
            if (arr[j].age > arr[j + 1].age) {  
                swap(&arr[j], &arr[j + 1]);  
            }  
        }  
    }  
}
```

```
int main() {  
    FILE *in_file;  
    Employee employees[100];  
    int count = 0;  
  
    // Create a dummy file  
    in_file = fopen("employee.txt", "w");  
    fprintf(in_file, "John 30 50000\n");  
    fprintf(in_file, "Jane 25 60000\n");  
    fprintf(in_file, "Peter 40 80000\n");
```

```

fprintf(in_file, "Alex 25 55000\n");
fclose(in_file);

in_file = fopen("employee.txt", "r");
if (in_file == NULL) {
    printf("Error opening input file.\n");
    return 1;
}

while (fscanf(in_file, "%s %d %f", employees[count].name, &employees[count].age,
&employees[count].salary) == 3) {
    count++;
}
fclose(in_file);

bubbleSort(employees, count);

printf("Sorted data by age:\n");
for (int i = 0; i < count; i++) {
    printf("%s %d %.2f\n", employees[i].name, employees[i].age, employees[i].salary);
}

return 0;
}

```

OR Q1) Perform the following operations using dynamic implementation of Queue:
i. Insert n elements in a queue. ii. Delete the element from the queue and display updated queue.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

```
struct Node *front = NULL, *rear = NULL;
```

```
void insert(int data) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    if (newNode == NULL) return;  
    newNode->data = data;  
    newNode->next = NULL;  
  
    if (rear == NULL) {  
        front = rear = newNode;  
    } else {  
        rear->next = newNode;  
        rear = newNode;  
    }  
}
```

```
void delete() {  
    if (front == NULL) {  
        printf("Queue is empty.\n");  
        return;  
    }  
}
```

```

    struct Node* temp = front;

    front = front->next;

    if (front == NULL) {
        rear = NULL;
    }

    printf("Deleted element: %d\n", temp->data);

    free(temp);
}

```

```

void display() {
    if (front == NULL) {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue elements: ");

    struct Node* temp = front;

    while (temp != NULL) {
        printf("%d ", temp->data);

        temp = temp->next;
    }

    printf("\n");
}

```

```

int main() {
    int n, i, element;

    printf("Enter the number of elements to insert: ");

    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
}

```

```
for (i = 0; i < n; i++) {  
    scanf("%d", &element);  
    insert(element);  
}  
  
delete();  
display();  
  
return 0;  
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Order(orderno, s_order_date, amount), Client(cno, name, address)

-- Relationship: A client can give one or more orders, but an order belongs to exactly one client.

-- Constraint: Order amount should be > 0

```
CREATE TABLE Client (  
    cno INT PRIMARY KEY,  
    name VARCHAR(50),  
    address VARCHAR(100)  
);
```

```
CREATE TABLE "Order" (  
    orderno INT PRIMARY KEY,
```

```
s_order_date DATE,  
amount DECIMAL(10, 2) CHECK (amount > 0),  
cno INT,  
FOREIGN KEY (cno) REFERENCES Client(cno)  
);
```

-- Insert records

```
INSERT INTO Client VALUES (1, 'John Doe', 'Pune');  
INSERT INTO Client VALUES (2, 'Jane Smith', 'Mumbai');  
INSERT INTO Client VALUES (3, 'Peter Jones', 'Delhi');  
INSERT INTO Client VALUES (4, 'Susan White', 'Mumbai');
```

```
INSERT INTO "Order" VALUES (101, '2024-05-15', 50000.00, 1);  
INSERT INTO "Order" VALUES (102, '2024-10-20', 120000.00, 2);  
INSERT INTO "Order" VALUES (103, '2024-10-25', 45000.00, 2);  
INSERT INTO "Order" VALUES (104, '2019-04-12', 35000.00, 4);  
INSERT INTO "Order" VALUES (105, '2024-09-01', 80000.00, 1);  
INSERT INTO "Order" VALUES (106, '2024-09-05', 90000.00, 1);
```

B. Execute the following queries in PostGreSQL. (Any 3).

- **i. Change order date of client number '04' to '12/4/19'.** (The prompt seems to refer to a client number, but updates are typically done on order numbers. Assuming a typo and using ordeno 104).

SQL

```
UPDATE "Order" SET s_order_date = '2019-04-12' WHERE ordeno = 104;
```

- **ii. Display all orders given in the month of October.**

SQL

```
SELECT * FROM "Order" WHERE EXTRACT(MONTH FROM s_order_date) = 10;
```

- **iii. Display all orders in descending order of amount.**

SQL

SELECT * FROM "Order" ORDER BY amount DESC;

- **iv. Delete all the clients of Mumbai.**

SQL

DELETE FROM "Order" WHERE cno IN (SELECT cno FROM Client WHERE address = 'Mumbai');

DELETE FROM Client WHERE address = 'Mumbai';

- **v. Count the number of orders having amount greater than 40000.**

SQL

SELECT COUNT(*) FROM "Order" WHERE amount > 40000;

C. Write a query to display the client name who has placed maximum number of orders.

SQL

SELECT C.name

FROM Client C

JOIN "Order" O ON C.cno = O.cno

GROUP BY C.name

ORDER BY COUNT(O.orderno) DESC

LIMIT 1;

OR C. Create a view to list names of clients having more than two sales orders.

SQL

CREATE VIEW Clients_With_More_Than_Two_Orders AS

SELECT C.name

FROM Client C

JOIN "Order" S ON C.cno = S.cno

GROUP BY C.cno, C.name

HAVING COUNT(S.orderno) > 2;

Slip 20

Section I: Data Structures

**Q1) Read the data from the 'employee.txt' file and sort on age using Merge sort.
Write the sorted data to another file 'sortedemponage.txt'.**

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
    char name[50];
```

```
    int age;
```

```
    float salary;
```

```
} Employee;
```

```
void merge(Employee arr[], int l, int m, int r) {
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    Employee L[n1], R[n2];
```

```
    for (i = 0; i < n1; i++)
```

```
        L[i] = arr[l + i];
```

```
    for (j = 0; j < n2; j++)
```

```
        R[j] = arr[m + 1 + j];
```

```
    i = 0;
```

```
    j = 0;
```

```
    k = l;
```

```

while (i < n1 && j < n2) {
    if (L[i].age <= R[j].age) {
        arr[k] = L[i];
        i++;
    } else {
        arr[k] = R[j];
        j++;
    }
    k++;
}

```

```

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

```

```

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

```

```

void mergeSort(Employee arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
    }
}

```

```

        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    FILE *in_file, *out_file;

    Employee employees[100];

    int count = 0;

    // Create a dummy employee.txt file
    in_file = fopen("employee.txt", "w");
    fprintf(in_file, "John 30 50000\n");
    fprintf(in_file, "Jane 25 60000\n");
    fprintf(in_file, "Peter 40 80000\n");
    fprintf(in_file, "Alex 25 55000\n");
    fclose(in_file);

    in_file = fopen("employee.txt", "r");
    if (in_file == NULL) {
        printf("Error opening input file.\n");
        return 1;
    }

    while (fscanf(in_file, "%s %d %f", employees[count].name, &employees[count].age,
&employees[count].salary) == 3) {
        count++;
    }
}

```

```

fclose(in_file);

mergeSort(employees, 0, count - 1);

out_file = fopen("sortedemponage.txt", "w");
if (out_file == NULL) {
    printf("Error opening output file.\n");
    return 1;
}

printf("Sorted data by age:\n");
for (int i = 0; i < count; i++) {
    printf("%s %d %.2f\n", employees[i].name, employees[i].age, employees[i].salary);
    fprintf(out_file, "%s %d %.2f\n", employees[i].name, employees[i].age,
employees[i].salary);
}

fclose(out_file);

return 0;
}

```

OR Q1) Write a C program to implement Static implementation of stack of integers with following operation: a. Initialize() b. Push() c. Pop() d. Isempty() e. Isfull() f. Peek().

C

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX_SIZE 100
```

```
int stack[MAX_SIZE];
```

```
int top = -1;
```

```
void Initialize() {
```

```
    top = -1;
```

```
    printf("Stack initialized.\n");
```

```
}
```

```
bool IsEmpty() {
```

```
    return top == -1;
```

```
}
```

```
bool IsFull() {
```

```
    return top == MAX_SIZE - 1;
```

```
}
```

```
void Push(int value) {
```

```
    if (IsFull()) {
```

```
        printf("Stack is full. Cannot push.\n");
```

```
    } else {
```

```
        stack[++top] = value;
```

```
        printf("%d pushed to stack.\n", value);
```

```
    }
```

```
}
```

```
int Pop() {
```

```
    if (IsEmpty()) {
```

```
        printf("Stack is empty. Cannot pop.\n");
```

```
        return -1;
    } else {
        return stack[top--];
    }
}
```

```
int Peek() {
    if (IsEmpty()) {
        printf("Stack is empty. No element to peek.\n");
        return -1;
    } else {
        return stack[top];
    }
}
```

```
void Display() {
    if (IsEmpty()) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack elements: ");
    for (int i = 0; i <= top; i++) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```
int main() {
```

```
Initialize();  
Push(10);  
Push(20);  
Push(30);  
printf("Peek element: %d\n", Peek());  
Display();  
printf("Popped element: %d\n", Pop());  
Display();  
return 0;  
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Customer (cust_no, cust_name, cust_city), Account (acc_no, acc_type, balance)

-- Account type can be either "Saving" or "Current"

-- Relationship: One-to-Many relationship between Customer and Account.

```
CREATE TABLE Customer (  
    cust_no INT PRIMARY KEY,  
    cust_name VARCHAR(50),  
    cust_city VARCHAR(50)  
);
```

```
CREATE TABLE Account (  
    acc_no INT PRIMARY KEY,
```

```
acc_type VARCHAR(10) CHECK (acc_type IN ('Saving', 'Current')),  
balance DECIMAL(10, 2),  
cust_no INT,  
FOREIGN KEY (cust_no) REFERENCES Customer(cust_no)  
);
```

-- Insert records

```
INSERT INTO Customer VALUES (1, 'Rajesh', 'Delhi');
```

```
INSERT INTO Customer VALUES (2, 'Priya', 'Ahilyanagar');
```

```
INSERT INTO Customer VALUES (3, 'Sanjay', 'Pune');
```

```
INSERT INTO Account VALUES (101123, 'Saving', 500000.00, 1);
```

```
INSERT INTO Account VALUES (101124, 'Current', 1200000.00, 2);
```

```
INSERT INTO Account VALUES (101125, 'Saving', 800000.00, 3);
```

```
INSERT INTO Account VALUES (101126, 'Saving', 1500000.00, 1);
```

B. Execute the following queries in PostgreSQL (any 3).

- **i. Find the names of all Customers staying in "Delhi".**

SQL

```
SELECT cust_name FROM Customer WHERE cust_city = 'Delhi';
```

- **ii. Give the total balance of all Saving accounts.**

SQL

```
SELECT SUM(balance) FROM Account WHERE acc_type = 'Saving';
```

- **iii. Find the maximum balance of an account.**

SQL

```
SELECT MAX(balance) FROM Account;
```

- **iv. Increase the balance of account no 101123 by Rs 10,000/-.**

SQL

```
UPDATE Account SET balance = balance + 10000 WHERE acc_no = 101123;
```

- v. Add column "AADHAR number" to customer table.

SQL

```
ALTER TABLE Customer ADD COLUMN aadhar_number VARCHAR(12);
```

C. Write a query to find the names and phone numbers of all Customers of Current accounts, having balance greater than 10,00,000/-. (The Customer table doesn't have a phone_no column. The query below will work if that column is added, or a temporary substitute is used).

SQL

```
SELECT C.cust_name, A.acc_no
FROM Customer C
JOIN Account A ON C.cust_no = A.cust_no
WHERE A.acc_type = 'Current' AND A.balance > 1000000;
```

OR C. Create a View to list details of all customers having saving account and not staying in city "Ahilyanagar".

SQL

```
CREATE VIEW Saving_Customers_Not_Ahilyanagar AS
SELECT *
FROM Customer C
JOIN Account A ON C.cust_no = A.cust_no
WHERE A.acc_type = 'Saving' AND C.cust_city != 'Ahilyanagar';
```

Slip 21

Section I: Data Structures

Q1) Perform the following operations using static implementation of Queue: i. Create a queue of n integers. ii. Delete the element from the queue and display updated queue.

C

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX_SIZE 100
```

```
int queue[MAX_SIZE];
```

```
int front = -1, rear = -1;
```

```
void createQueue(int n) {
```

```
    int value;
```

```
    if (n > MAX_SIZE) {
```

```
        printf("Queue size exceeds maximum limit.\n");
```

```
        return;
```

```
    }
```

```
    front = rear = -1;
```

```
    printf("Enter %d integers:\n", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &value);
```

```
        if (rear == MAX_SIZE - 1) {
```

```
            printf("Queue is full.\n");
```

```
            return;
```

```
        }
```

```
        if (front == -1) {
```

```
            front = 0;
```

```
        }
```

```
        rear++;
```

```
        queue[rear] = value;
```

```
    }
```

```
}
```

```
void deleteElement() {
```

```
if (front == -1) {  
    printf("Queue is empty.\n");  
    return;  
}  
  
int deleted_value = queue[front];  
printf("Deleted element: %d\n", deleted_value);  
  
if (front == rear) {  
    front = -1;  
    rear = -1;  
} else {  
    front++;  
}  
}
```

```
void display() {  
    if (front == -1) {  
        printf("Queue is empty.\n");  
        return;  
    }  
  
    printf("Updated queue: ");  
    for (int i = front; i <= rear; i++) {  
        printf("%d ", queue[i]);  
    }  
  
    printf("\n");  
}
```

```
int main() {  
    int n;
```

```

printf("Enter the number of integers for the queue: ");
scanf("%d", &n);
createQueue(n);
deleteElement();
display();
return 0;
}

```

OR Q1) Write a menu driven program for the following operations on singly linked list: i. Insert an element in a linked list in a particular position. ii. Search an element in a linked list.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node* next;
};

```

```
struct Node* head = NULL;
```

```

void insertAtPosition(int data, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) return;
    newNode->data = data;
    newNode->next = NULL;

    if (position == 1) {
        newNode->next = head;
    }
}

```

```
    head = newNode;
    return;
}
```

```
struct Node* temp = head;
for (int i = 1; i < position - 1 && temp != NULL; i++) {
    temp = temp->next;
}
```

```
if (temp == NULL) {
    printf("Invalid position.\n");
    free(newNode);
    return;
}
```

```
newNode->next = temp->next;
temp->next = newNode;
}
```

```
void searchElement(int value) {
    int position = 1;
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == value) {
            printf("Element %d found at position %d.\n", value, position);
            return;
        }
        temp = temp->next;
    }
}
```

```
        position++;  
    }  
    printf("Element %d not found in the list.\n", value);  
}
```

```
void displayList() {  
    if (head == NULL) {  
        printf("List is empty.\n");  
        return;  
    }  
    printf("Linked list: ");  
    struct Node* temp = head;  
    while(temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
int main() {  
    int choice, data, position;  
  
    printf("Creating a sample list (10, 20, 30):\n");  
    insertAtPosition(10, 1);  
    insertAtPosition(20, 2);  
    insertAtPosition(30, 3);  
    displayList();  
}
```

```
do {  
    printf("\n--- Singly Linked List Operations ---\n");  
    printf("1. Insert an element\n");  
    printf("2. Search for an element\n");  
    printf("3. Exit\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch (choice) {  
        case 1:  
            printf("Enter element to insert: ");  
            scanf("%d", &data);  
            printf("Enter position to insert: ");  
            scanf("%d", &position);  
            insertAtPosition(data, position);  
            displayList();  
            break;  
        case 2:  
            printf("Enter element to search: ");  
            scanf("%d", &data);  
            searchElement(data);  
            break;  
        case 3:  
            printf("Exiting program.\n");  
            break;  
        default:  
            printf("Invalid choice. Please try again.\n");  
    }  
}
```

```
} while (choice != 3);  
  
return 0;  
}
```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A. Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Bus (bno, capacity, depot_name), Driver (dno, driver_name, license_no, address, age, salary)

-- Relationship: Bus and Driver are related with Many to Many relationships with the descriptive attribute Date_of_duty.

```
CREATE TABLE Bus (  
    bno INT PRIMARY KEY,  
    capacity INT,  
    depot_name VARCHAR(50)  
);
```

```
CREATE TABLE Driver (  
    dno INT PRIMARY KEY,  
    driver_name VARCHAR(50),  
    license_no VARCHAR(20),  
    address VARCHAR(100),  
    age INT,  
    salary DECIMAL(10, 2)  
);
```

```

CREATE TABLE Bus_Driver (
    bno INT,
    dno INT,
    Date_of_duty DATE,
    PRIMARY KEY (bno, dno, Date_of_duty),
    FOREIGN KEY (bno) REFERENCES Bus(bno),
    FOREIGN KEY (dno) REFERENCES Driver(dno)
);

```

-- Insert records

```
INSERT INTO Bus VALUES (101, 50, 'Kothrud Depot');
```

```
INSERT INTO Bus VALUES (102, 45, 'Hadapsar Depot');
```

```
INSERT INTO Bus VALUES (103, 60, 'Swargate Depot');
```

```
INSERT INTO Driver VALUES (1, 'Swapnil', 'DL12345', 'Pune', 28, 45000.00);
```

```
INSERT INTO Driver VALUES (2, 'Suresh', 'DL67890', 'Mumbai', 24, 40000.00);
```

```
INSERT INTO Driver VALUES (3, 'Sameer', 'DL11223', 'Pune', 35, 50000.00);
```

```
INSERT INTO Driver VALUES (4, 'Amit', 'DL44556', 'Nashik', 22, 35000.00);
```

```
INSERT INTO Bus_Driver VALUES (101, 1, '2017-01-15');
```

```
INSERT INTO Bus_Driver VALUES (102, 2, '2017-01-15');
```

```
INSERT INTO Bus_Driver VALUES (101, 3, '2009-02-12');
```

```
INSERT INTO Bus_Driver VALUES (103, 4, '2009-02-12');
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Find the number of buses having capacity more than 20.**

SQL

```
SELECT COUNT(*) FROM Bus WHERE capacity > 20;
```

- **ii. Count number of drivers who did duty on "15/01/2017".**

SQL

```
SELECT COUNT(DISTINCT dno) FROM Bus_Driver WHERE Date_of_duty = '2017-01-15';
```

- **iii. Display the license number of all drivers whose name starts with 'S'.**

SQL

```
SELECT license_no FROM Driver WHERE driver_name LIKE 'S%';
```

- **iv. Calculate the average salary of a driver.**

SQL

```
SELECT AVG(salary) FROM Driver;
```

- **v. Find all details of driver named "Swapnil".**

SQL

```
SELECT * FROM Driver WHERE driver_name = 'Swapnil';
```

C. Write a query to find the names of all drivers who were driving buses from "Kothrud Depot" on 12/2/2009.

SQL

```
SELECT d.driver_name
```

```
FROM Driver d
```

```
JOIN Bus_Driver bd ON d.dno = bd.dno
```

```
JOIN Bus b ON bd.bno = b.bno
```

```
WHERE b.depot_name = 'Kothrud Depot' AND bd.Date_of_duty = '2009-02-12';
```

OR C. Create a view to find all details of buses which are driven by drivers whose age is less than 25 years.

SQL

```
CREATE VIEW Young_Driver_Buses AS
```

```
SELECT DISTINCT B.*
```

```
FROM Bus B
```

```
JOIN Bus_Driver BD ON B.bno = BD.bno
```

```
JOIN Driver D ON BD.dno = D.dno
```

```
WHERE D.age < 25;
```

Slip 22

Section I: Data Structures

Q1) Read the data from the "employee.txt" file and sort on age using Quick sort and write the sorted data to another file 'sortedemponage.txt'.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
    char name[50];
```

```
    int age;
```

```
    float salary;
```

```
} Employee;
```

```
void swap(Employee* a, Employee* b) {
```

```
    Employee t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
int partition(Employee arr[], int low, int high) {
```

```
    int pivot_age = arr[high].age;
```

```
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {
```

```
        if (arr[j].age <= pivot_age) {
```

```
            i++;
```

```
            swap(&arr[i], &arr[j]);
```

```

    }
}
swap(&arr[i + 1], &arr[high]);
return (i + 1);
}

```

```

void quickSort(Employee arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

```

int main() {
    FILE *in_file, *out_file;

    Employee employees[100];

    int count = 0;

    // Create a dummy employee.txt file
    in_file = fopen("employee.txt", "w");
    fprintf(in_file, "John 30 50000\n");
    fprintf(in_file, "Jane 25 60000\n");
    fprintf(in_file, "Peter 40 80000\n");
    fprintf(in_file, "Alex 25 55000\n");
    fclose(in_file);

    in_file = fopen("employee.txt", "r");

```

```

if (in_file == NULL) {
    printf("Error opening input file.\n");
    return 1;
}

while (fscanf(in_file, "%s %d %f", employees[count].name, &employees[count].age,
&employees[count].salary) == 3) {
    count++;
}
fclose(in_file);

quickSort(employees, 0, count - 1);

out_file = fopen("sortedemponage.txt", "w");
if (out_file == NULL) {
    printf("Error opening output file.\n");
    return 1;
}

printf("Sorted data by age:\n");
for (int i = 0; i < count; i++) {
    printf("%s %d %.2f\n", employees[i].name, employees[i].age, employees[i].salary);
    fprintf(out_file, "%s %d %.2f\n", employees[i].name, employees[i].age,
employees[i].salary);
}
fclose(out_file);

return 0;
}

```

OR Q1) Write a menu driven program for the following operations on singly linked list: i. Insert an element in a linked list in a particular position. ii. Delete a particular element from a linked list.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* head = NULL;
```

```
void insertAtPosition(int data, int position) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    if (newNode == NULL) return;
```

```
    newNode->data = data;
```

```
    newNode->next = NULL;
```

```
    if (position == 1) {
```

```
        newNode->next = head;
```

```
        head = newNode;
```

```
        return;
```

```
    }
```

```
    struct Node* temp = head;
```

```
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
```

```
        temp = temp->next;
```

```
}
```

```
if (temp == NULL) {  
    printf("Invalid position.\n");  
    free(newNode);  
    return;  
}
```

```
newNode->next = temp->next;  
temp->next = newNode;  
}
```

```
void deleteElement(int key) {  
    struct Node *temp = head, *prev = NULL;  
  
    if (temp != NULL && temp->data == key) {  
        head = temp->next;  
        free(temp);  
        printf("Element %d deleted.\n", key);  
        return;  
    }
```

```
while (temp != NULL && temp->data != key) {  
    prev = temp;  
    temp = temp->next;  
}
```

```
if (temp == NULL) {
```

```

        printf("Element %d not found.\n", key);
        return;
    }

    prev->next = temp->next;
    free(temp);
    printf("Element %d deleted.\n", key);
}

void displayList() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked list: ");
    struct Node* temp = head;
    while(temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, data, position;

    printf("Creating a sample list (10, 20, 30):\n");
    insertAtPosition(10, 1);

```

```
insertAtPosition(20, 2);
```

```
insertAtPosition(30, 3);
```

```
displayList();
```

```
do {
```

```
    printf("\n--- Singly Linked List Operations ---\n");
```

```
    printf("1. Insert an element\n");
```

```
    printf("2. Delete an element\n");
```

```
    printf("3. Exit\n");
```

```
    printf("Enter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch (choice) {
```

```
        case 1:
```

```
            printf("Enter element to insert: ");
```

```
            scanf("%d", &data);
```

```
            printf("Enter position to insert: ");
```

```
            scanf("%d", &position);
```

```
            insertAtPosition(data, position);
```

```
            displayList();
```

```
            break;
```

```
        case 2:
```

```
            printf("Enter element to delete: ");
```

```
            scanf("%d", &data);
```

```
            deleteElement(data);
```

```
            displayList();
```

```
            break;
```

```
        case 3:
```

```

        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice. Please try again.\n");
    }
} while (choice != 3);

return 0;
}

```

Section II: Database Management Systems

Q 2) PostgreSQL Queries

A.Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Employee(eno,ename, designation, salary), Department(dno, dname, location)

-- Relationship: Employee-Department: One to Many.

-- Constraint: employee salary greater than 5000

```

CREATE TABLE Department (
    dno INT PRIMARY KEY,
    dname VARCHAR(50),
    location VARCHAR(50)
);

```

```

CREATE TABLE Employee (
    eno INT PRIMARY KEY,
    ename VARCHAR(50),

```

```
designation VARCHAR(50),  
salary DECIMAL(10, 2) CHECK (salary > 5000),  
dno INT,  
FOREIGN KEY (dno) REFERENCES Department(dno)  
);
```

-- Insert records

```
INSERT INTO Department VALUES (10, 'IT', 'Nanded');
```

```
INSERT INTO Department VALUES (20, 'HR', 'Pune');
```

```
INSERT INTO Department VALUES (30, 'Sales', 'Nanded');
```

```
INSERT INTO Employee VALUES (101, 'Sagar', 'Developer', 65000.00, 10);
```

```
INSERT INTO Employee VALUES (102, 'Priya', 'Manager', 80000.00, 20);
```

```
INSERT INTO Employee VALUES (103, 'Amit', 'Analyst', 55000.00, 10);
```

```
INSERT INTO Employee VALUES (104, 'Anjali', 'Sales Rep', 62000.00, 30);
```

```
INSERT INTO Employee VALUES (105, 'Aniket', 'Analyst', 58000.00, 10);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Change designation of employee "Sagar" to Manager.**

SQL

```
UPDATE Employee SET designation = 'Manager' WHERE ename = 'Sagar';
```

- **ii. Display the minimum salary of an employee.**

SQL

```
SELECT MIN(salary) FROM Employee;
```

- **iii. List the details of all the departments located at Nanded city.**

SQL

```
SELECT * FROM Department WHERE location = 'Nanded';
```

- **iv. Display the details of employees whose names starts with A.**

SQL

SELECT * FROM Employee WHERE ename LIKE 'A%';

- **v. List the department number wise count of employees.**

SQL

SELECT dno, COUNT(*) FROM Employee GROUP BY dno;

C. Write a query to list the department name-wise total salary.

SQL

SELECT d.dname, SUM(e.salary)

FROM Department d

JOIN Employee e ON d.dno = e.dno

GROUP BY d.dname;

OR C. Create a view to display the department details having maximum employees.

SQL

CREATE VIEW Dept_Max_Employees AS

SELECT d.dname

FROM Department d

JOIN Employee e ON d.dno = e.dno

GROUP BY d.dname

ORDER BY COUNT(e.eno) DESC

LIMIT 1;

Slip 23

Section I: Data Structures

Q1) Write a menu driven program for the following operations on doubly linked list:

i. Create and display linked list. ii. Reverse a linked list and display it.

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* prev;
```

```
    struct Node* next;
```

```
};
```

```
struct Node* createList(int n) {
```

```
    struct Node* head = NULL;
```

```
    struct Node* newNode, *temp;
```

```
    int data;
```

```
    if (n <= 0) return NULL;
```

```
    printf("Enter %d data elements:\n", n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
        if (newNode == NULL) break;
```

```
        printf("Enter element %d: ", i + 1);
```

```
        scanf("%d", &data);
```

```
        newNode->data = data;
```

```
        newNode->prev = NULL;
```

```
        newNode->next = NULL;
```

```
        if (head == NULL) {
```

```
            head = newNode;
```

```

    } else {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}
return head;
}

```

```

void displayList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Doubly linked list elements: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

void reverseList(struct Node** head_ref) {
    struct Node* temp = NULL;

```

```

struct Node* current = *head_ref;
while (current != NULL) {
    temp = current->prev;
    current->prev = current->next;
    current->next = temp;
    current = current->prev;
}
if (temp != NULL) {
    *head_ref = temp->prev;
}
}

int main() {
    struct Node* head = NULL;
    int choice, n;

    do {
        printf("\n--- Doubly Linked List Operations ---\n");
        printf("1. Create and Display\n");
        printf("2. Reverse and Display\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter the number of nodes: ");
                scanf("%d", &n);

```

```

        head = createList(n);

        displayList(head);

        break;
    case 2:

        if (head == NULL) {

            printf("List is empty. Create a list first.\n");

            break;

        }

        reverseList(&head);

        displayList(head);

        break;
    case 3:

        printf("Exiting program.\n");

        break;
    default:

        printf("Invalid choice. Please try again.\n");

    }
} while (choice != 3);

return 0;
}

```

OR Q1) Perform the following operations using static implementation of stack: i. Accept n integers from user and push it into the stack. ii. Pop the element from the stack and display it (Use isEmpty()).

C

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
int isEmpty() {
```

```
    return top == -1;
```

```
}
```

```
void push(int item) {
```

```
    if (top >= MAX - 1) {
```

```
        printf("Stack Overflow!\n");
```

```
        return;
```

```
    }
```

```
    stack[++top] = item;
```

```
}
```

```
int pop() {
```

```
    if (isEmpty()) {
```

```
        printf("Stack Underflow!\n");
```

```
        return -1;
```

```
    }
```

```
    return stack[top--];
```

```
}
```

```
int main() {
```

```
    int n, item;
```

```
    printf("Enter the number of integers to push: ");
```

```
    scanf("%d", &n);
```

```
printf("Enter %d integers:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &item);
    push(item);
}

printf("Popping elements from the stack:\n");
while (!isEmpty()) {
    printf("%d\n", pop());
}

return 0;
}
```

Section II: Database Management Systems

Q2) PostgreSQL Queries

A.Create the above database in PostGreSQL and insert sufficient records.

SQL

-- Database Schema: Customer (cno, cust_name, cust_city), Account (ano,acc_type, balance)

-- Relationship: One to Many relationships between Customer and Account.

-- Constraint: Account type Should be either "Saving" or "Current"

```
CREATE TABLE Customer (
    cno INT PRIMARY KEY,
    cust_name VARCHAR(50),
    cust_city VARCHAR(50)
```

);

```
CREATE TABLE Account (  
    ano INT PRIMARY KEY,  
    acc_type VARCHAR(10) CHECK (acc_type IN ('Saving', 'Current')),  
    balance DECIMAL(10, 2),  
    cno INT,  
    FOREIGN KEY (cno) REFERENCES Customer(cno)  
);
```

-- Insert records

```
INSERT INTO Customer VALUES (1, 'Rohan', 'Pune');
```

```
INSERT INTO Customer VALUES (2, 'Nitin', 'Mumbai');
```

```
INSERT INTO Customer VALUES (3, 'Sahil', 'Delhi');
```

```
INSERT INTO Customer VALUES (4, 'Pravin', 'Pune');
```

```
INSERT INTO Account VALUES (101, 'Saving', 50000.00, 1);
```

```
INSERT INTO Account VALUES (102, 'Current', 150000.00, 2);
```

```
INSERT INTO Account VALUES (103, 'Saving', 200000.00, 3);
```

```
INSERT INTO Account VALUES (104, 'Saving', 75000.00, 4);
```

```
INSERT INTO Account VALUES (105, 'Saving', 90000.00, 1);
```

B. Execute the following queries in PostGreSQL (any 3).

- **i. Count the number of customers whose name contains substring "in".**

SQL

```
SELECT COUNT(*) FROM Customer WHERE cust_name LIKE '%in%';
```

- **ii. Give the average balance of all Current accounts.**

SQL

```
SELECT AVG(balance) FROM Account WHERE acc_type = 'Current';
```

- **iii. Display details of accounts in sorted order of balance.**

SQL

```
SELECT * FROM Account ORDER BY balance;
```

- **iv. Increase the balance of all accounts of customer named as "Sahil" by Rs 10000.**

SQL

```
UPDATE Account SET balance = balance + 10000 WHERE cno = (SELECT cno FROM Customer WHERE cust_name = 'Sahil');
```

- **v. Add column "phone_no" to customer table.**

SQL

```
ALTER TABLE Customer ADD COLUMN phone_no VARCHAR(15);
```

C. Write a query to list the city-wise customer names having minimum balance.

SQL

```
SELECT c.cust_city, c.cust_name
```

```
FROM Customer c
```

```
JOIN Account a ON c.cno = a.cno
```

```
WHERE (c.cust_city, a.balance) IN (
```

```
    SELECT cust_city, MIN(a2.balance)
```

```
    FROM Customer c2
```

```
    JOIN Account a2 ON c2.cno = a2.cno
```

```
    GROUP BY cust_city
```

```
);
```